

Developing an Intelligent Chatbot: Technical Report

Group 2: 4320891*, 100491318 and 100493742

May 22, 2025

Abstract

The following technical report describes the successful completion of the Artificial Intelligence coursework two project. It evidences our methods and selection of languages, packages and tools, outlines our initial work plan and our overall approach to organising work throughout the project. This includes how the project software was designed and developed, implemented, tested and evaluated. Finally, this report discusses potential issues with our project and possible future work that was not possible to complete within the project time-frame.

1 Introduction

The objective of the project was to design and develop an intelligent chatbot system, leveraging skills and knowledge in artificial intelligence and software engineering acquired throughout this and other modules. The software system should be capable of (a) finding the cheapest available train ticket between two train stations in the UK, (b) improving customer service by predicting train delay for trains between Norwich and London Liverpool Street, and (c) assisting train operational staff by handling rail blockage contingencies occurring between Colchester and Norwich. These minimum requirements should be met through continued and consistent dialogue with the user through a user interface (UI).

1.0.1 Intelligent Chatbots

An intelligent chatbot is a software system that aims to simulate a human-like conversation with the user through using artificial intelligence techniques, such as natural language processing (NLP) (IBM 2021). ‘Intelligent’ refers to the chatbot’s ability to process a range of unpredictable user input through the use of NLP techniques, rather than relying on preset questions and responses (Radziwill & Benton 2017, Chizhik & Zhrebtsova 2020). This typically entails a chatbot using NLP techniques to extract a user intent and any relevant linguistic entities, which stimulate a response from a knowledge base related to the organisation’s business logic.

The central aim of developing intelligent chatbots is to enhance user experience through seamlessly generating related and meaningful responses to a range of unpredictable user input. In addition, they offer a method of low running cost, 24/7 customer service, virtual assistance for employees, and the ability to capture useful customer data, all of which aids businesses (Abedin et al. 2022). They are commonly used in a wide range of applications, including customer service assistance, marketing, financial applications, and e-commerce, amongst others (Lin et al. 2023).

1.0.2 Contributions

The following students contributed to this project: 4320891, 100491318 and 100493742. We would like to thank the lecturers and laboratory staff on their helpful and constructive feedback throughout the project.

1.1 Background and Motivation

1.1.1 Motivation

A central motivation for this project was the lack of a real-time train delay prediction application for UK rail networks. Whilst an application with the aim of predicting train delays before they occur has been recently brought to the market (Stoate 2025), to the best of the authors' knowledge, a journey delay prediction application does not currently exist. That is, an application which can inform the user with a prediction of how long their current journey is delayed by, and therefore, how long it might take for them to arrive at their destination. This would be useful in a wide range of scenarios, for instance, where the user is commuting and would like to know how late they might be to a work meeting, or when the user is getting a train to the airport and would like to know whether they are likely to miss their flight.

In addition, by combining train delay prediction into a wider software system that also includes ticket purchasing and contingency plans for operational staff, the project aims to pull together three important elements of rail chatbots into one software system. This could ultimately improve customer and employee service by containing all the necessary information about UK rail travel in one application.

1.1.2 Coursework Specification

The project involved three main objectives which were broken down into tasks (Wang 2018):

- Task 1 - Find the cheapest available rail ticket for the user. This involves using the chatbot to gather information from the user on an intended journey and using this to find the cheapest available ticket. The information about this ticket should be sent to the chat with a link to purchase the ticket on the ticketing site. This component of the chatbot should be able to make complex bookings involving outgoing and return journeys, with railcards, children, and other booking features.
- Task 2 - Predict train time delay between Norwich and London Liverpool Street and any other additional customer service functions. This involves adding more functions to the knowledge base of the chatbot system to improve customer services. The central addition is a train journey time delay predictor, that aims to inform the user of the predicted arrival time of their train if they are currently delayed. This should only predict delay on lines between Norwich and London Liverpool Street.
- Task 3 - Assist train operational staff by giving information on rail blockage contingencies occurring between Colchester and Norwich. This involves developing an expert system (integrated into the chatbot) to handle rail track blockage events between Colchester and Norwich. The aim of this system is to inform operational staff about appropriate actions and with relevant knowledge given a particular operational event occurring. As such, this system should be accessible only by operational staff and requires a separate staff login through the chatbot UI.

Additionally, testing should be carried out on the chatbot system, and a presentation and demonstration prepared, alongside the production of this technical report.

1.2 Coursework Objective

The main objective of this coursework is to develop an intelligent chatbot software system that can respond to users or operational staff using NLP techniques to find the cheapest available rail ticket between two UK destinations, use artificial intelligence to predict train journey time delay between Norwich and London Liverpool Street, as well as inform operational staff about handling line blockage events between Colchester and Norwich.

1.3 Difficulties and Risks

There are a number of difficulties and risks relating to the project. These include:

- Legality around sourcing rail data. For instance, ensuring the proper approval for using historic rail data to train models has been given, or ensuring the legality of scraping ticket information from a particular website. Although National Rail and Greater Anglia websites were suggested for scraping data in the coursework brief (Wang 2018), these sites state plainly in their terms-of-use that web scraping is not allowed on their webpages. We thereby, initially opted to use ‘www.traintickets.com’ to scrape ticket data (see: ‘scraper.py’ in ‘chatbot_scripts’), as the terms of use do not forbid web scraping. However, after communication with the manager of the National Rail Online Journey Planner data feed, we were able to set-up access to the feed which is the true origin of all live rail ticket information in the UK.
- Misleading the user. It is possible that some users believe that the chatbot interaction is interaction with a real person. This could mislead the user and lead to a poor customer experience (Radziwill & Benton 2017). It is important in this respect that the user is informed of the nature of the conversation from the beginning, and that, if possible, they are given the option to chat to a real person instead.
- Threat of data leakage. Either for sensitive user data, such as operational staff logins, or for business data such as journey delay predictions. Data can be leaked in many ways, including through the chatbot itself. Steps should be taken to mitigate this. For instance, our application uses environment files to safely store any keys and passwords.
- Risk of inaccurate or unreliable results to user inputs. For instance, this might include giving unreliable ticket information that leads the user to make a purchase they do not want. It might also include unreliable delay predictions, inconveniencing the user. This might cause serious brand damage if it is a regular occurrence. Resultantly, although the chatbot supplies the cheapest and most appropriate ticket information for the user, it does not purchase the ticket for them. Instead, it directs the user to the ticket booking page of the appropriate ticket provider, in order that the user has the final decision on ticket booking.
- Issues around intellectual property use. For instance, if the models or software libraries being used are not open-source, this is an intellectual property infringement. The terms and conditions of use of all libraries and other pre-made software used in the chatbot must be checked to ensure they are available for the specific use. For instance, Meta’s Wit.ai NLP platform, which is used in the project to handle text intent and entity recognition, is completely open source and free for commercial use.
- Threats to user confidentiality. It is possible that in recording a user-chatbot conversation, confidential user information is being stored. The chatbot system must be designed in a way that aims to prevent this. We decided, in this regard, not to develop a database for user conversation history. However, we understand that storing conversation histories could improve customer service. Therefore, future work could include the development of a database to store a user’s conversation history with suitable measures in place to protect their data.

1.4 Work Plan

We used an agile work methodology throughout the project, with weekly meetings where possible to discuss progress and objectives for the following week. We also used a Trello board to share information, including meeting notes and links to the technical report and GitHub project repository.

Project Stage	week 7	week 8	week 9	EB week 1	EB week 2	EB week 3	EB week 4	week 10	week 11	week 12	final week
<i>Planning</i>											
<i>Research</i>											
<i>Development</i>											
<i>Testing</i>											
<i>Technical Report</i>											
<i>Presentation</i>											

Figure 1: Gantt chart showing the proposed project workflow. Week refers to the UEA week number. EB refers to Easter Break week number.

2 Related Work

In designing and developing our intelligent chatbot software system, we studied related work on chatbot architecture. This enabled us to understand the type of chatbot we were developing, the flow of the software system and the limitations of our approach.

Modern intelligent chatbot systems can be divided into three types based on how chatbot responses are generated: (a) rule-based, (b) retrieval-based, and (c) generative-based. Rule-based chatbots analyse a user input to match certain linguistic patterns to pre-defined response templates. These are closed-domain and goal-oriented (Luo et al. 2022). Retrieval-based use text-vectorization and semantic similarity scores to match user inputs with best possible responses in a database. These are used for both closed-domain and open-domain cases, and can be goal-oriented or chat-oriented (Chizhik & Zhrebtsova 2020). Generative-based approaches are typically used for chatbots that handle a wide range of possible functions and are open-domain, such as virtual personal assistants or advanced chatbots, such as ChatGPT. These generate new responses, instead of selecting best matches from a database, with the most advanced bots employing generative pre-trained transformer architecture.

Regardless of the response generation mechanism, modern chatbots share some common architecture. At a high level, they require:

1. A user interface (UI). Whether this is textual, in the case of most systems, or speech-based, in the case of many virtual personal assistants (such as Apple’s Siri or Amazon’s Alexa).
2. A natural language understanding (NLU) component. This uses NLP techniques to parse a textual or speech input into a structured semantic representation, from which the logic, meaning, intent and sentiment of the utterance can be obtained (Adamopoulou & Moussiades 2020). This can consist of topic detection, tokenisation, parts-of-speech-tagging (POS), text normalisation, intent analysis and entity linking, amongst others. For instance, intent analysis uses similarity scoring to determine the intent of an input. That is, how similar the input intent is to some stored utterances with predefined intent. Entity linking usually consists of disambiguation, named entity recognition (NER) and template selection. The central one of these techniques is NER, which allows key entities to be recognised and labelled in the input.
3. A dialogue manager component. This handles the processed user input from the NLU component to generate a response based on the requirements and logic of the chatbot software system (Mohammed & Aref 2022). The dialogue manager often has a context tracking component. This enables the chatbot to keep track of the state of a particular user’s conversation and, if required, retrieve or store user states and conversations from a knowledge base (Chizhik & Zhrebtsova 2020).
4. A response generation component. This uses the strategies from the dialogue manager module to generate a textual or text-to-speech response (Luo et al. 2022). For open domain generative-based chatbots, this could be a natural language generation (NLG) module. However, not all chatbots require this functionality. For closed-domain chatbots, rule-based strategies can be sufficient for giving question/answer style responses. The key difference between the approaches to designing the response generation component is that an NLG module can generate novel responses based on user input, whereas rule-based and retrieval-based strategies have a fixed set of responses.

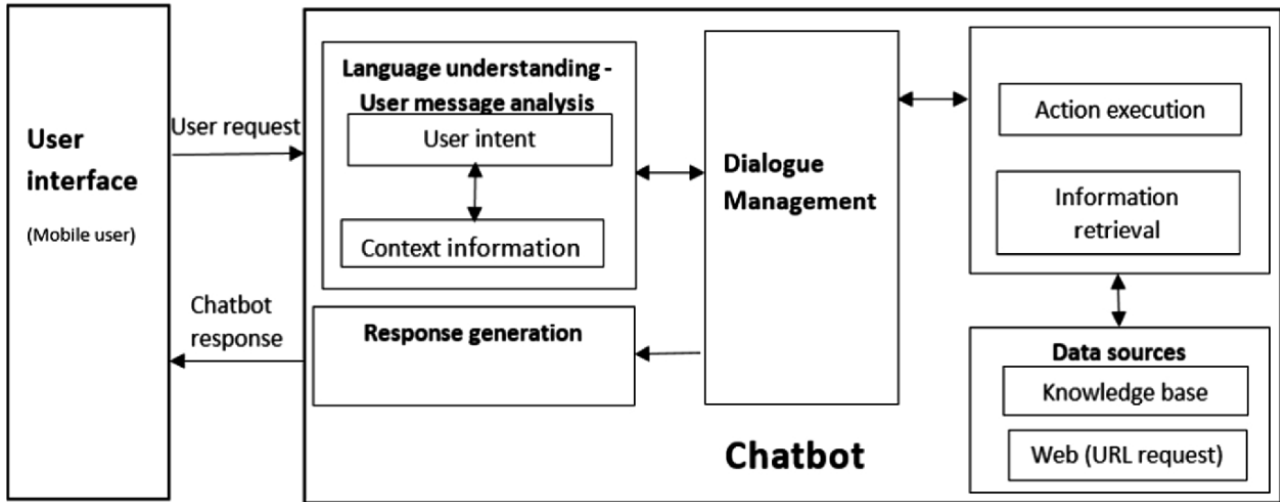


Figure 2: Figure showing a typical intelligent chatbot architecture. In this case, developed as a mobile application. Taken from (Adamopoulou & Moussiades 2020).

The type of chatbot system we were asked to develop is a task-based, closed-domain system, to be used for several specific tasks relating to UK rail travel. It should employ AI techniques to enable it to properly understand a user input and to predict train journey delays. It should also apply online retrieval techniques for finding up-to-date information. However, it need not employ generative-based response strategies. It is, therefore, a rule-based intelligent chatbot architecture. It employs intelligent learning techniques to understand and manage dialogue, but largely employs rule-based response strategies.

3 Methods, Tools and Frameworks

The following section details the methods, programming languages, packages, tools and frameworks that we used in the development of our intelligent chatbot software system.

3.1 Methods

3.1.1 User Interface

We developed a text-based user interface (UI) using a React.js frontend framework, which is served as a web application using Vite. We chose React.js due to its simple and component-based architecture, its extensive libraries and because it uses a virtual document object model (DOM) meaning that components are efficiently updated. Redux was used to update and store the application state. Given that we developed an operational staff login function to enable only operational staff to ask for rail contingency plans, we needed a central store for a login token. The Axios library was used to handle request/response logic, due to its simple-to-use syntax and high browser compatibility. TailwindCSS and the DaisyUI plugin were used to provide styling due to the ease of use, professional look and theme control.

For added accessibility, the UI includes both text-based user input and speech-to-text input. Including a speech-to-text function is important for those unable to easily use a keyboard and for situations in which speaking to the application is easier, such as when walking. As part of this functionality, if a user inputs speech, the application automatically responds using text to speech synthesis.

3.1.2 Natural Language Processing

Natural Language Processing (NLP) was performed using Wit.ai and served through an application programming interface (API). We begin the project using Python libraries (SpaCy and BeautifulSoup) to perform NLP. However, the results of intent and entity recognition were poor when using a range of

utterance structures and colloquial language. Therefore, we chose to use Meta’s Wit.ai NLP platform to handle the NLP component of the chatbot. Wit.ai is a free to use and open-source NLP platform, that allows intent and entity recognition, can deal with both text and voice input, and is served through an API.

We created a new Wit.ai model and trained it to recognise certain intents and entities relating to the different tasks of the project. For instance, we trained it to recognise a range of different kinds of booking request, delay prediction requests and requests for contingency plan information, amongst others. During training, we deliberately used a variety of phrases in order that the model could pickup more nuanced and idiosyncratic language use. This included UK regional language use, which can have a different pragmatics to the prima facie syntactic-semantic complex.¹ This enabled our Wit.ai model to return structured JSON data clearly defining the intent and entities of a user input posted in a request to the API. Moreover, Wit.ai has a Python library that offers seamless integration.

3.1.3 Reasoning Engine

We developed a reasoning engine to deal with determining what actions to perform based on the user’s intent. This was developed in a rule-based way using Python’s Experta expert system framework. We used Experta as it is based on the well-known and reliable CLIPS expert system, but using Python syntax. We used an expert system rather than a series of functions and conditional statements for the reasoning engine, as the rules can be triggered in different orders, reducing time and memory requirements, and the system is simpler and can be extended more readily. For similar reasons, we also used an Experta expert system to order the logic in the operational event subsystem for task 3.

3.1.4 Ticket Booking

Due to legal issues around web scraping National Rail or Greater Anglia website data, we initially opted to build a web scraping component that scraped ticket data from ‘trainticket.com’ using a series of functions in ‘scraper.py’. This used the Python library Selenium with a headless browser to automatically fill-in the website ticket query forms and return the three tickets closest to the time requested by the user. A separate module called ‘cheapest_ticket.py’ was used to find the cheapest of these three tickets, which was sent to the chat.

Due to limitations of the web scraping module in handling complex and possibly erroneous queries, particularly regarding incorrect station names, we opted to use the National Rail Online Journey Planner data-feed as the final method of providing ticketing information. This required communication with the data-feed manager to acquire login credentials. The Online Journey Planner feed is the ultimate source of live fare information for all rail ticket booking websites in the UK and is free to use if the number of requests do not exceed several thousand per month. The data-feed is provided through a live web service description language (WSDL) file which accepts HTTP requests using a simple object access protocol (SOAP) envelope. This means that access to the data is fast and seamless, with responses noticeably faster than when using web scraping techniques. Extracting useful data from the feed response was performed using XML element tree extraction techniques. Again, although much of the logic for extracting the specific ticket request is performed in the ‘feed_access.py’ file after the data-feed response has been received, this is still a substantially faster and less resource intensive method than using web scraping.

As mentioned in Section 1.3 we made the decision not to directly book the ticket for the user, but rather give the user all the information they need to book the cheapest and most appropriate journey, as well as a link to the relevant ticket booking website.

3.1.5 Train Delay - Predictive Model Data Preprocessing, Training and Model Selection

The workflow of the delay prediction task included data preprocessing, extensive feature engineering, training multiple regression models (Linear Regression, Random Forest, XGBoost, LightGBM, Cat-

¹Consider how the utterance ‘Alright’, which in many areas of the UK simply means ‘Hello’, could be misinterpreted as a question or as an affirmation.

Boost) using a segment-wise approach and model evaluation.

Initial steps - Preprocessing steps included removing a duplicate column in the '2024_service_details.csv' dataset, verifying consistency in the columns across the three datasets before concatenation into a single data-frame, dealing with missing values and filtering the data-frame by removing cancelled trains and specific rail ID (RID) values to leave a shape of (212,174,10). The 'arrival_delay' column was selected as the target variable.

Feature engineering and data cleaning - Feature engineering was performed on the data-frame to create predictive features which are grouped into several categories as follows:

- Target variable calculation - 'planned_arrival_time' and 'actual_arrival_time' were converted to minutes-since-midnight using 'time_to_minutes'. 'arrival_delay' was calculated as 'arrival_delay' = 'actual_arrival_minutes' - 'planned_arrival_minutes'.
- Time-based features -
 - Planned Arrival Minutes - Time of planned arrival in minutes.
 - Day of Week - Extracted from 'date_of_service' (0 = Monday, 6 = Sunday).
 - Year - Extracted from 'date_of_service'.
 - Is Weekend - Binary feature (1 if 'day_of_week' is 5 or 6, else 0).
 - Hour of Day - Derived from 'planned_arrival_minutes' divided by 60.
 - Is Peak Hour - Binary feature (1 if 'hour_of_day' is 7-9 or 16-18, else 0).
 - Is Holiday - Binary feature (1 if 'date_of_service' matches a UK public holiday from a predefined list for 2022–2024, else 0).
 - Time Since Journey Start - Difference between 'planned_arrival_minutes' and the first station's 'planned_arrival_minutes' for each RID.
- Station-based features -
 - Station Order - Sequential order of stations in a journey (cumcount() + 1 per RID).
 - Station Index - Label-encoded location using LabelEncoder to assign numerical values to stations.
 - Is First Station - Binary feature (1 if 'station_order' = 1, else 0).
 - Is Last Station - Binary feature (1 if 'station_order' equals the maximum 'station_order' for the RID, else 0).
- Delay-based features -
 - Previous Arrival Delay - Lagged 'arrival_delay' from the previous station in the same RID (filled with 0 for first stations).
 - Cumulative Delay - Sum of 'arrival_delay' up to the current station, adjusted to exclude the current station's delay.
 - Previous Delays (Lag Features) - 'prev_delay_1' for delay at the previous station and 'prev_delay_2' for delay at the station two stops prior (both filled with 0 for early stations).
- Route-based features -
 - Journey length - total number of stations in each RID (maximum 'station_order').
 - Planned Travel Time to Next - Difference in 'planned_arrival_minutes' between consecutive stations (filled with 0 for last stations).
 - Actual Travel Time to Next - Difference in 'actual_arrival_minutes' between consecutive stations (filled with 0 for last stations).

- Interaction features - Station-Day Interaction - product of 'station_order' and 'day_of_week' to capture combined effect.

The data was further cleaned to remove the unnecessary columns: 'date_of_service', 'location', 'planned_arrival_time', 'planned_departure_time', 'actual_departure_time', 'late_canc_reason'. Moreover, the specific RID values '202405237619353' and '202408237668818' were removed.

Feature selection - The following features were, shown with coefficients, were selected for the initial round of model training using statistical summary from statsmodels, as shown in Figure 3: 'previous_arrival_delay' (0.394), 'prev_delay_1' (0.394), 'prev_delay_2' (0.1978), 'is_last_station' (-0.2668), 'is_peak_hour' (-0.1175), 'is_first_station' (-0.0985), 'day_of_week' (-0.0232), 'is_weekend' (-0.0512), 'station_day_interaction' (0.0031). These features demonstrated the highest positive and negative coefficients, which indicates closer relationships with the target variable, and thereby, a greater possibility of these features impacting a model's predictive capacity.

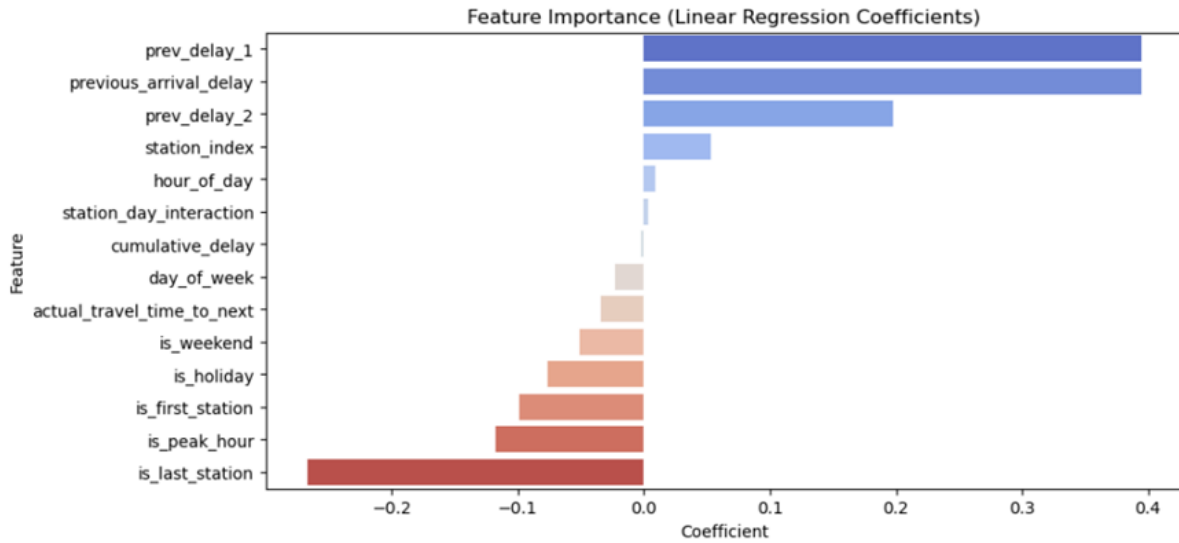


Figure 3: Bar plot depicting feature importance showing the linear regression coefficients.

Later additions to engineered features After the initial round of model training further feature engineering was undertaken in an attempt to prevent apparent model over-fitting. The prediction data was expanded with more hand-crafted features, which are shown below:

- **Convert Times & Flag Cancellations:** Turn 'HH:MM' into minutes and tag any journey that had a late cancellation.
- **Date & Holiday Info:** Extract day, month, weekday/weekend, and flag public holidays.
- **Travel Times & Rush Hours:** Compute durations between stops and identify morning/evening peak times.
- **Station Details:** Track the sequence of stops, assign numeric station IDs, and include average station delays.
- **Past & Rolling Delays:** Incorporate delays from the previous one or two stops and their rolling averages.
- **Cyclic & Season Flags:** Encode hours and days using sine and cosine transformations to capture cyclic patterns; one-hot encode seasons.
- **Clean & Transform Target:** Clip extreme delay values, apply log transformation to the target, and drop unused raw features.

- **Data Split & Cleanup:** Split data by trip ID, remove highly correlated features, and impute missing values.

The new features encode elements of the data such as recent delay history (lagged and rolling means), station-specific effects, holiday/peak flags and cyclic time components. This gives the models direct, high-signal inputs that explain much of the minute-by-minute variation in delays. Trees (RF, XGBoost, LightGBM, CatBoost) are then able to carve-up the feature space more effectively reducing error by over 70% compared to the previous features, which can be seen in Figures 4 and 5 for the root mean square error (RMSE) and R^2 scores, respectively.

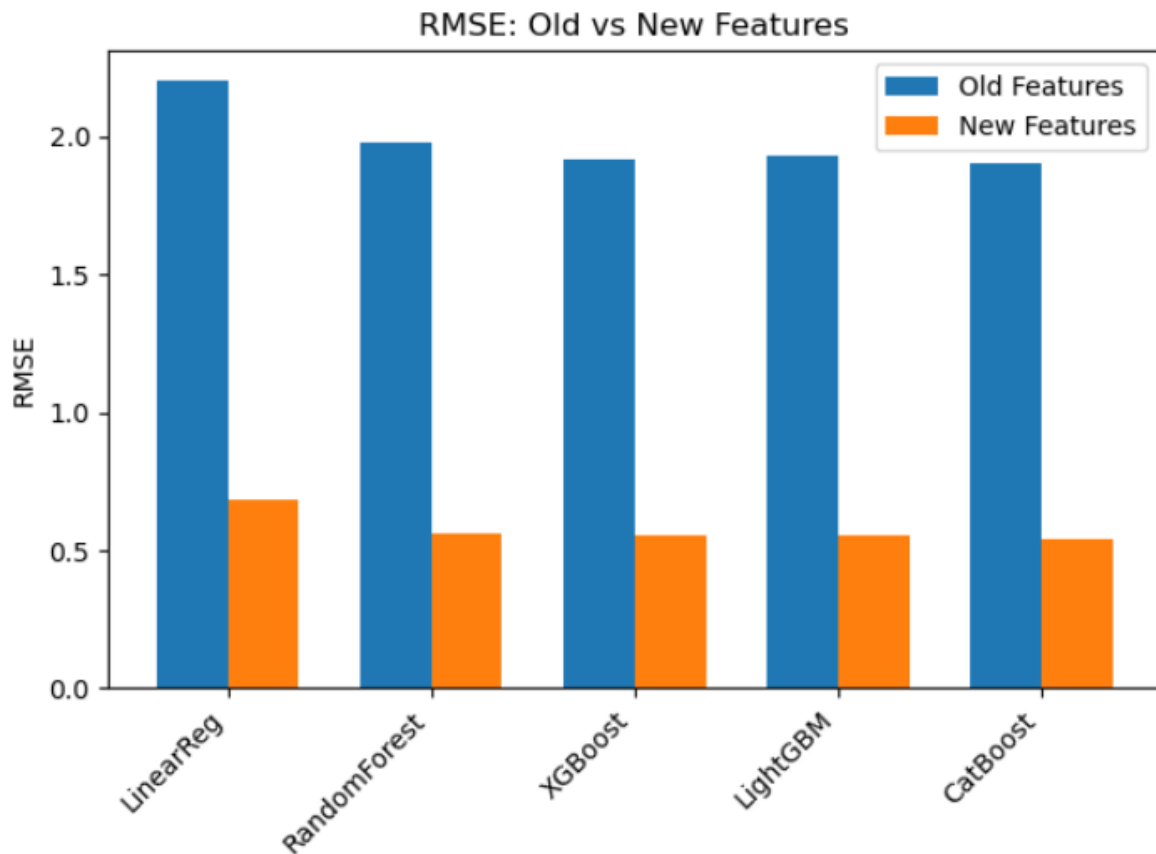


Figure 4: Bar plot showing the RMSE scores of the linear regression, random forest, XGBoost, LightGBM and CatBoost models trained on the first (old) and second (new) iterations of the data feature set.

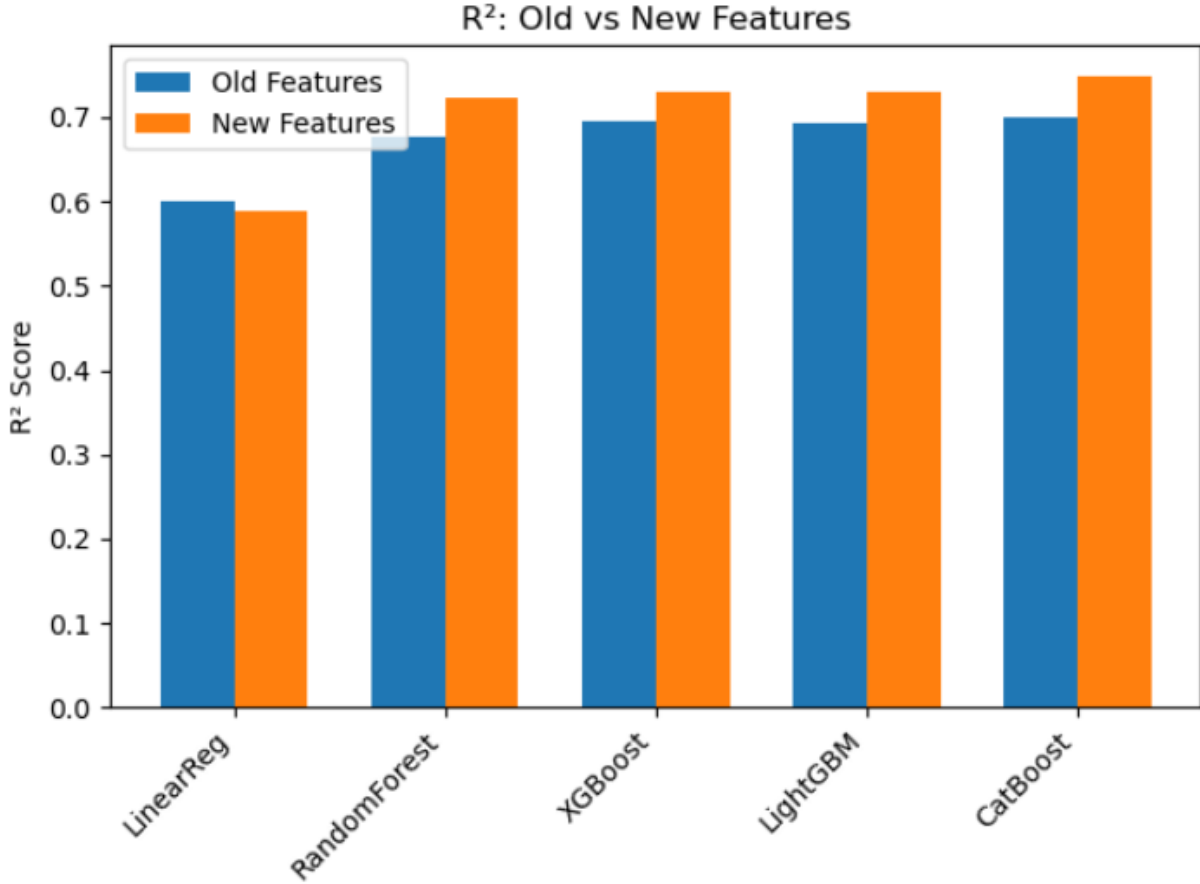


Figure 5: Bar plot showing the R^2 scores of the linear regression, random forest, XGBoost, LightGBM and CatBoost models trained on the first (old) and second (new) iterations of the data feature set.

A SHAP summary was plotted for each of the models, using Shapley values to exhibit feature importance. Figure 6 shows the SHAP summary for the CatBoost model (as this is the model which was selected as the best-performing), with the most influential predictors of arrival delay as the previous stop's delay ('prev_delay_1'), cancellation flag ('late_canc') and the travel-time ratio ('tt_ratio'), with the higher values driving larger predicted delays.

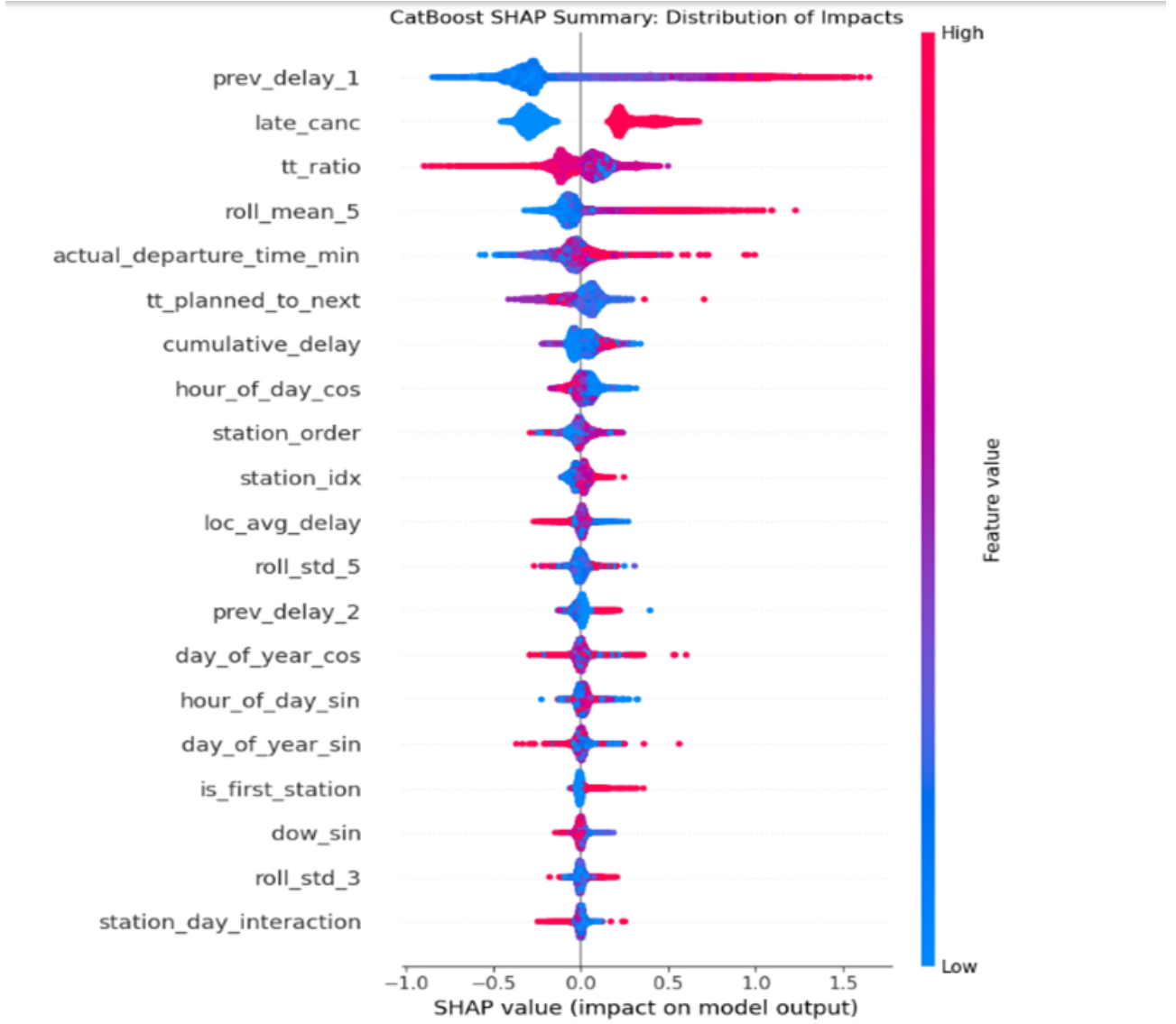


Figure 6: Figure showing Shapley values for the CatBoost regression model with additional hand-crafted features.

Model training - A train/test split of 80/20 was performed by RID to ensure entire journeys were in either the training or test set, so as not to bias the model. Multiple regression models were then trained on the data with engineered features. These were linear regression, random forest regressor, XGBoost regressor, lightGBM regressor and CatBoost regressor.

Models were trained using a segment-wise modelling strategy. This involved training separate models for each station-to-station segment, defined as the current station index to the next station index. Segments with insufficient data (<10 for linear regression and <5 for XGBoost) were skipped. A function was written to perform chained prediction, which predicts delays for future stations by iteratively applying segment models, updating delay-related features (e.g. 'cumulative_delay' and 'prev_delay_1') after each prediction.

Model evaluation - Models were evaluated using the RMSE and the R^2 or coefficient of determination. RMSE measures the average magnitude of the prediction error and was calculated using the following equation:

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2}$$

R^2 depicts the variance in the dependent variable and was calculated using the following equation:

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Results and discussion - The results, given in Table 1 and depicted visually in Figures 7 and 8, show that the CatBoost regression model performs marginally better than other models with an RMSE of 0.50 and an R^2 of 0.73. The linear regression model performs worst overall with an RMSE of 0.69 and an R^2 of 0.59, whilst the random forest, XGBoost and LightGBM models all exhibit similar scores.

Model	RMSE	R^2 Score
Linear Regression	0.69	0.59
Random Forest	0.55	0.70
XGBoost	0.54	0.71
LightGBM	0.55	0.72
CatBoost	0.50	0.73

Table 1: Table showing the overall performance comparison of the different regression models and approaches used to predict train journey delay time.

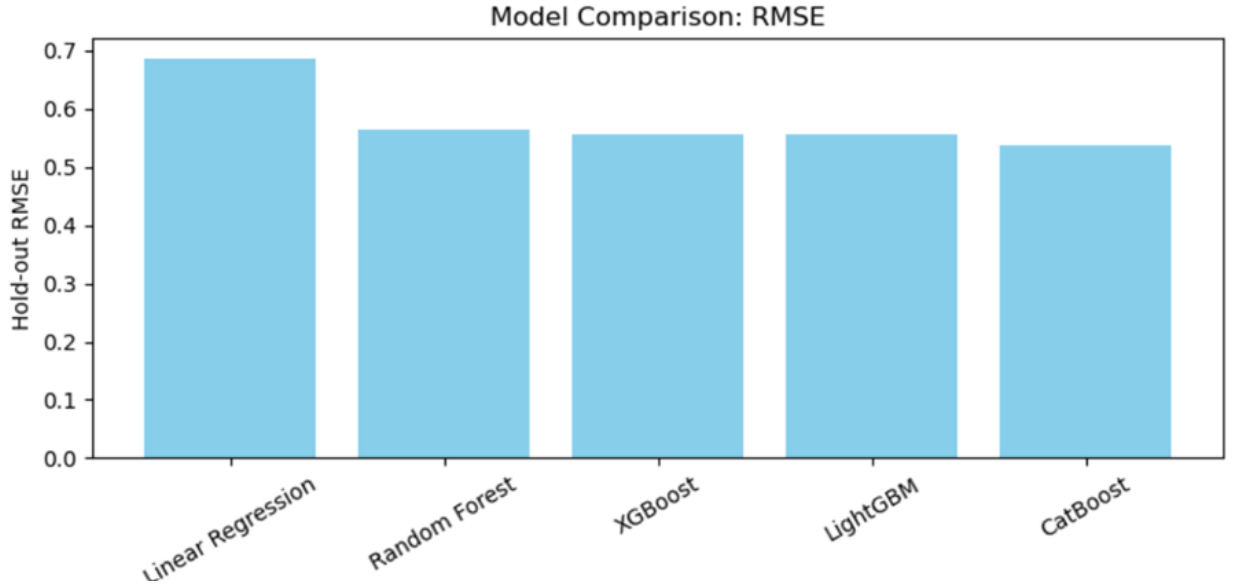


Figure 7: Bar plot showing the RMSE scores for linear regression, random forest, XGBoost, LightGBM and CatBoost models when tested on the hold-out set.

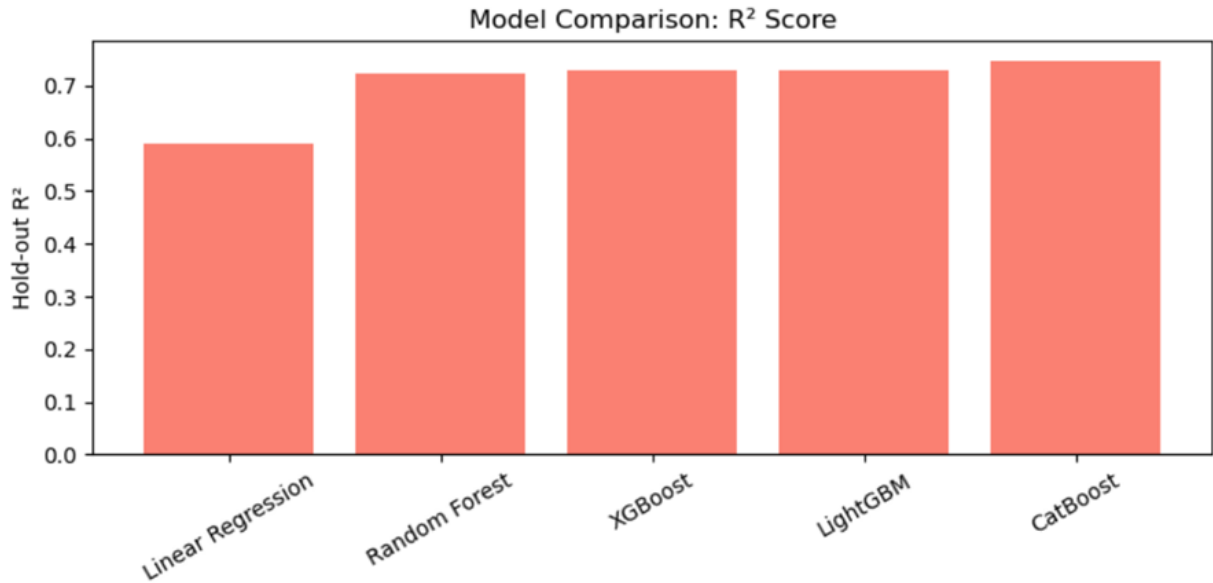


Figure 8: Bar plot showing the R^2 scores for linear regression, random forest, XGBoost, LightGBM and CatBoost models when tested on the hold-out set.

The CatBoost model may perform better than the other models with relation to the metrics used, due to its capacity to automatically handle numerical and categorical variables, and its native ordered boosting with built-in regularization, which makes it especially adept at modelling the complex, noisy patterns in train delays, whilst converging quickly. However, XGBoost and LightGBM exhibited RMSE and R^2 that were not significantly lower than the CatBoost model. These models use gradient-boosting frameworks to grow level-wise or leaf-wise trees, and thereby capture non-linear interactions not encapsulated in our feature engineering. Random Forest provides a baseline result by averaging many decorrelated trees to reduce variance while linear regression’s linear assumptions limit its ability to model cascading delay effects and feature interactions.

The segment-wise approach, where separate models are trained for each station-to-station segment (e.g. NRW $\rightarrow$ DIS, DIS $\rightarrow$ SMK), was chosen because of its ability to capture localized factors such as track conditions, traffic density, geographical elements and operational practices between each station pair. This means that by defining segments as ‘station_index’ $\rightarrow$ ‘next_station_index’ (e.g., 0.0 $\rightarrow$ 12.0), the approach ensures that each model is tailored to the unique characteristics of a station pair. In addition, since train journeys involve multiple stations, delays propagate from one segment to the next. A single global model combines delay patterns from all segments, which may weaken the unique signals of individual segments. The segment-wise approach empowers chained predictions where the predicted delay for one segment is used as input (‘previous_arrival_delay’, ‘cumulative_delay’) for the next, allowing end-to-end journey delay forecasting. Therefore, segment-wise models allow for more precise predictions by focusing on the unique data distribution of each segment. Furthermore, it enhances model performance, ‘explainability’, and flexibility that allow better predictions for each route.

3.1.6 Contingency Plan Retrieval and Knowledge Base

As mentioned above, an Experta expert system was developed to handle task 3, in which operational staff users of the chatbot make operational event queries to retrieve contingency plan information. The intertwining logic of operational events and contingency plans was extracted from several historic pdfs supplied for the project, and used to develop a conversation flow that could capture the relevant event information from a staff user of the chatbot to build a query and return the necessary contingency plan information. This system acts as a subsystem (sub-engine) to the reasoning engine, given that the reasoning engine conducts dialogue flow based on user intent. As a subsystem, it requires engine halting in order to query missing information from the user and salience to order the triggering of

rules and manage the dialogue more effectively.

Rules	Action(s)	Storage	Extraction Method
OperationalEvent (location)	Triggers a question or declares an event	Station code embeddings (JSON)	SentenceTransformer + cosine similarity
OperationalEvent (event)	Triggers ‘BlockageType(partial/full)’	None	None
BlockageType (partial/full)	Triggers either ‘TimetableAction(amend)’ or ‘TimetableAction(cancel)’	None	None
TimetableAction (amend)	Adds message. Triggers ‘GoldenServices(yes)’ and ‘InformChelmsfordSupervisor(yes)’ if applicable	Amendments (vector DB)	FIASS
TimetableAction (cancel)	Adds message. Also triggers ‘GoldenServices(yes)’, ‘InformChelmsfordSupervisor(yes)’, ‘SpecialStopOrders(yes)’, ‘StablingRequirements(yes)’	Cancellations (vector DB)	FIASS
GoldenServices (yes)	Adds message to final response	None	None
InformChelmsford Supervisor(yes)	Adds message to final response	None	None
SpecialStopOrders (yes)	Finds cancelled headcodes and adds stop order message	Stop order rules (vector DB)	FIASS
Stabling Requirements(yes)	Uses headcodes and time to retrieve stabling needs	Stabling data (vector DB)	FIASS
OperationalEvent (current_delay)	Triggers: - ‘HoldConnections(yes)’ - ‘WeatherEvent(trigger)’ - ‘DeclareCLS2(yes)’ if delay ≥ 30 min - ‘AlternativeTransport(organise)’ if delay ≥ 60 min	None	None
Alternative Transport(organise)	Adds message to response	None	None
DeclareCLS2(yes)	Adds message to response	None	None

Rules	Action(s)	Storage	Extraction Method
InformCaterers(yes)	Adds message to response	None	None
HoldConnections (yes)	Adds connection info for af- fected stations	Connection matrix (vector DB)	FIASS
WeatherEvent (trigger)	Calls weather API and adds message	None	Open Meteo API via ‘weather.py’
FinalResponse (trigger)	Aggregates and returns final messages	Session state array	None

Table 2: Table showing the operational events subsystem rules, actions, storage and extraction methods in ‘operational_events.py’. ‘FIASS’ stands for ‘Facebook AI Similarity Search’.

The operational events subsystem combines user answers to questions with internal knock-on rules to logically extract information from the knowledge base and build a final response, which is returned as a message to the chat. Table 2 gives a comprehensive list of the rules, actions, storage and extraction methods used in the ‘operational_events.py’ subsystem. The subsystem extracts the relevant information from vector embedding files using the ‘Facebook AI Similarity Search’ (FIASS) library, JSON files and a series of functions, and returns instructions for different staff members, similar to the layout in the written contingency plans in ‘D3_AICC_05Contingency Plan Manual.pdf’. An example of the subsystem logic is given in Figure 9. This depicts a staff user query for a full line blockage between Norwich and Diss with a current delay of around 15 minutes.

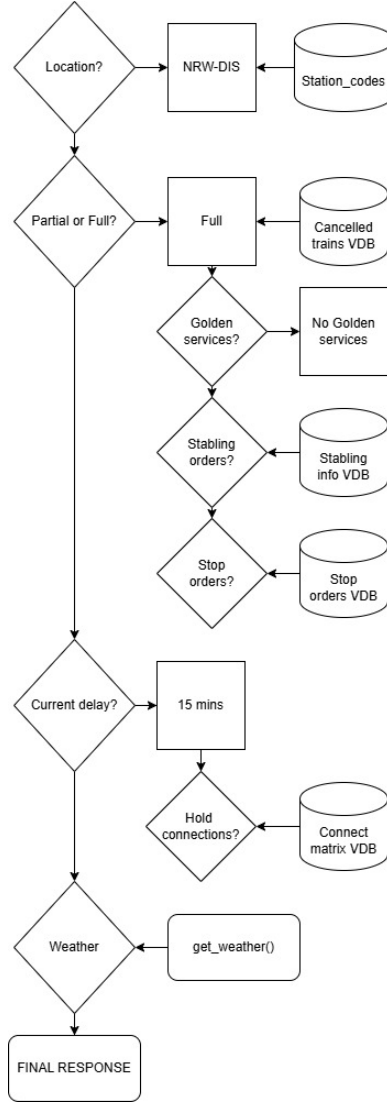


Figure 9: Flow diagram depicting the logic of an example staff contingency plan query to the contingency plan subsystem, where VDB stands for ‘vector database’. This shows the flow through the subsystem and points at which information is retrieved from the knowledge base - in this case, station code vector embeddings held in a JSON file, the vector files, and through the Open Meteo weather API.

OpenAI was used for text-extraction and text-vectorisation, as it performs well with both structured and unstructured data, allowing varied and implied information to be captured. A set of ‘.npz’ vector files produced using OpenAI’s ‘Text-embedding-3-small’ model, as well as JSON files, were chosen to store the relevant contingency plan information from the historic pdfs. These were then indexed using the FIASS library to allow high-speed nearest-neighbor lookup. In order to preserve data granularity and context, each dataset was indexed separately ensuring that queries returned the most relevant matches from the correct source. The FIASS library was considered appropriate for similarity search as it allows optimised search of a large quantity of information, including high dimensionality vector embeddings. This is due to the use of GPU backend acceleration, multi-threading and advanced search techniques.

3.2 Languages, Packages, Tools

Table 3 gives a high-level overview of the languages, libraries and packages used to develop different components in our AI chatbot software system.

Component	Language(s)	Libraries/Packages
UI	JavaScript	Tailwind CSS, Redux, Axios, js-cookie, React, PostCSS, DaisyUI, Vite
NLP	Python	FastAPI (WebSockets, JSONResponse), Wit (Wit.ai integration)
Reasoning Engine	Python	Pydantic (request/response schemas), Experta (expert system logic)
Web Scraper / Data Feed Accessor	Python	Selenium, requests, xml
Delay Predictor	Python	FastAPI, Pydantic, Pandas, XGBoost, Joblib (model/encoder loading), Logging (error handling)
Operational Event Subsystem	Python	Experta, FastAPI, Open_Meteo, Pandas, OpenAI, FIASS, Numpy
Databases	SQL / Python	PostgreSQL (for staff login) and '.npy' files for vector embeddings

Table 3: Table giving an overview of the languages, packages and libraries used to design and develop our AI chatbot system components.

3.3 Development Frameworks

As mentioned above, the UI component of the chatbot was built in the React.js development framework due to its simple and component-based architecture, its extensive libraries and because it uses a virtual DOM meaning that components are efficiently updated. As previously detailed, Wit.ai was used as the NLP platform, as it offers an effective framework for training and developing NLP models. The Locust load-testing framework was used for scalability evaluation.

4 Design of the Chatbot

The following section describes the design of our AI chatbot software system.

4.1 MoSCoW analysis

The initial design phase of our chatbot involved gathering system requirements. This enabled us to set meaningful work towards achieving the coursework objectives. As part of this, we performed a MoSCoW analysis, shown in Figure 10. This enabled us to determine the high-level structure of the chatbot by deciding what the system must have, could have, should have and won't have. We also conducted a literature review, given in Section 2, in order to better understand how our chatbot system fits within the wider field.

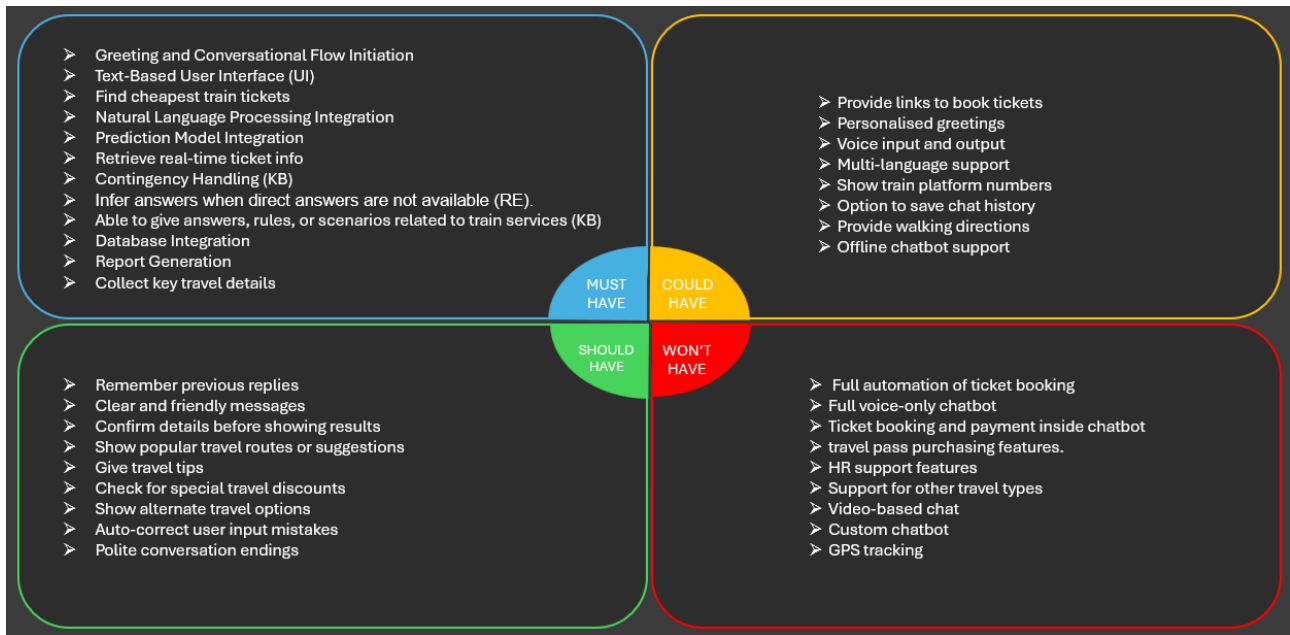


Figure 10: MoSCoW analysis for intelligent chatbot software system, conducted in the initial design phase showing elements and features the software system must have, could have, should have and won't have.

4.2 The Architecture of the Chatbot

Our initial design phase fed into a more detailed set of architectural designs for our intelligent chatbot system. These included a high level component architecture design of the entire chatbot system (Figure 11), as well as file-level feature component designs of the frontend (Figure 12) and the backend system (Figure 13). These component architecture designs were used to determine the approximate file structure, which varies slightly from the final implementation. Classes, functions and interactions were determined based on the blueprint in Figures 12-13 during the project implementation.

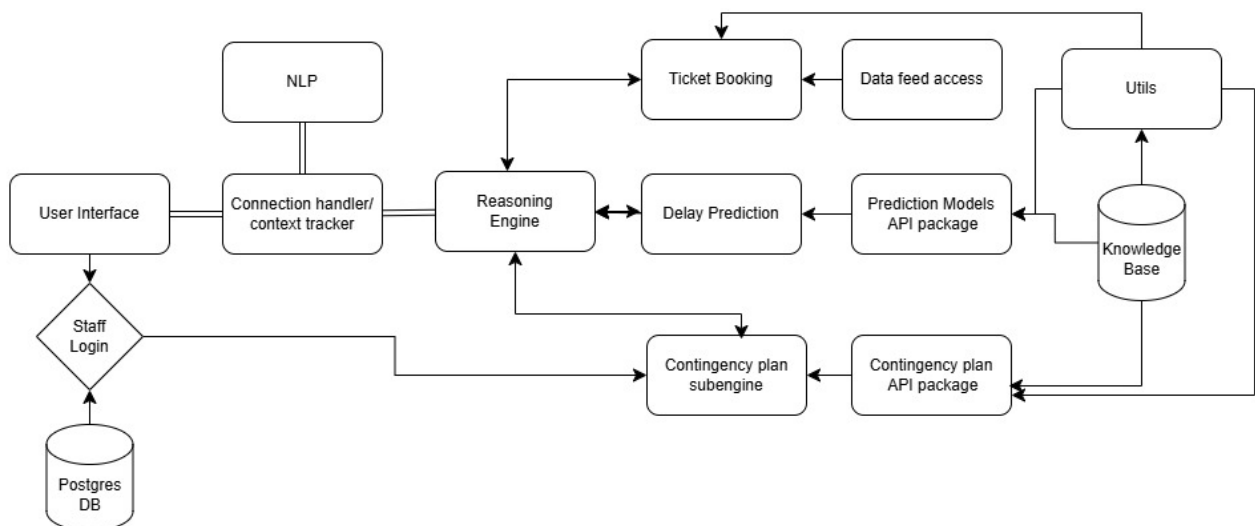


Figure 11: Diagram showing the high level component architecture of our intelligent chatbot software system. This is intended to give the reader an overview of the system architecture, although it differs slightly from the final implementation.

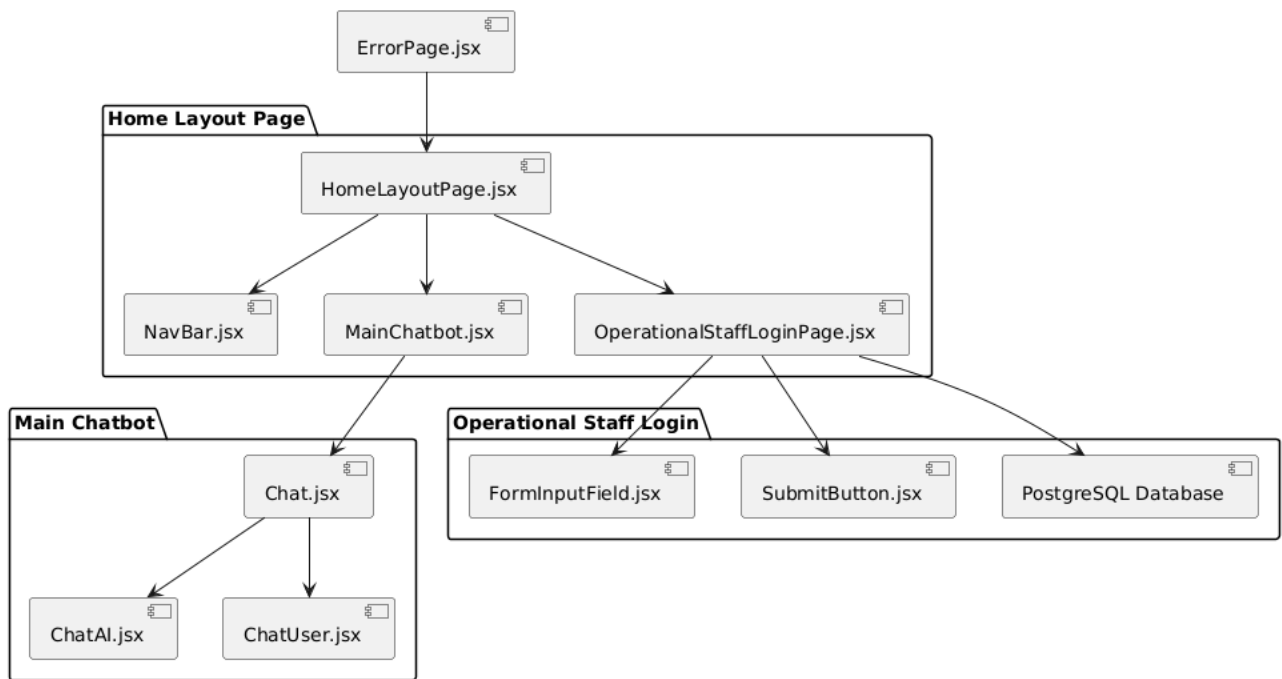


Figure 12: Feature component diagram depicting main file structure of the chatbot software system frontend built in React.js

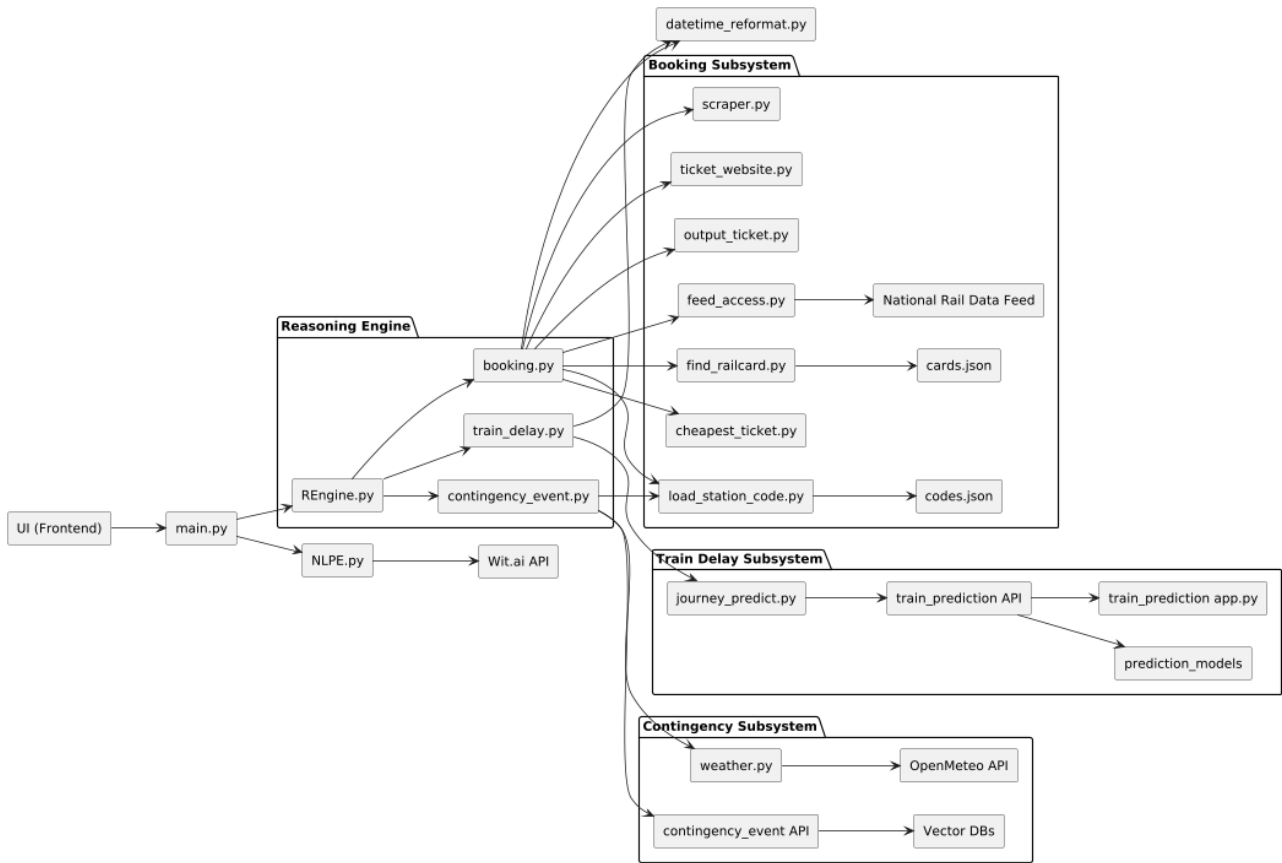


Figure 13: Feature component diagram depicting main file structure of the chatbot software system backend built principally in Python. It should be noted that in the final file structure ‘Booking Subsystem’ was not included as a file name. Instead, most of the files shown to be in this subsystem in this figure were included in a folder called ‘utils’. Moreover, ‘contingency_event.py’ was more appropriately named ‘operational_events.py’, and the contingency API endpoint was served through the ‘train-prediction-api’ package for simplicity.

4.3 Dialogue Flow

Due to the central importance of dialogue flow in our AI chatbot software system, we include our initial plan for dialogue flow (Table 4). This depicts how user intent relates to the software system files, and the flow of the response. For instance, if the user intent is ‘greeting’, the system will generate a simple pre-set string response. However, if the intent is ‘booking’, the system will initiate a question and answer dialogue to obtain the necessary information to make a ticket booking from the user. It is important to note that a session state acts as context tracking in order to hold this information between API calls between the frontend and backend of the application. In addition, the user can abort the state at any stage during the dialogue.

Intent	System	Dialogue
greeting	main.py, NLPE.py, REngine.py	Simple response
thanks	main.py, NLPE.py, REngine.py	Simple response
booking	main.py, NLPE.py, REngine.py, booking.py	Q&A response: - datetime out? - IF return, datetime return? - OR add return? - origin station? - destination station? - IF unmentioned, ticket type? - no. passengers? (e.g. adults, children) - IF railcard, railcard type?
train_delay	main.py, NLPE.py, REngine.py, train_delay.py	Q&A response: - datetime out? - origin station? - destination station? - current delay? - current station?
operational_event	main.py, NLPE.py, REngine.py, operational_event.py	Q&A response: - location? (stations between or line section) - event type? (e.g. full or partial) - current delay?
goodbye	main.py, NLPE.py, REngine.py	Simple response

Table 4: Table depicting dialogue flow through our chatbot system. This shows how user intent affects the software system and the nature of the dialogue response.

5 Implementation

The following section builds on Sections 3 and 4 in detailing the implementation of our AI chatbot software system.

5.1 User Interface

The UI component was written in React.js, as outlined above, and has the following pages and components:

- Pages - ‘HomeLayoutPage.jsx’, ‘MainChatbot.jsx’, ‘OperationalStaffLoginPage.jsx’ and ‘ErrorPage.jsx’. ‘HomeLayoutPage.jsx’ is used to preserve the same look and feel of the web application across pages. The ‘MainChatbot.jsx’ page displays user and chatbot messages (Figure 14). The ‘ErrorPage.jsx’ is used as a redirect if a routing error occurs. ‘OperationalStaffLoginPage.jsx’ allows staff users to login allowing them to query contingency plans. The login function is handled via an API call to the ‘train-prediction-api’ package, ‘main.py’ file, that accesses a PostgreSQL database. On successful authentication a ‘true’ boolean value is returned and stored in a central state store using Redux. This is passed to the chatbot backend to allow contingency plan access.

- Components - 'Chat.jsx' is a parent component of 'ChatAI.jsx' and 'ChatUser.jsx'. The latter child components allow the user and AI chat messages to be handled differently. 'FormInputField.jsx' handles staff login details. 'NavBar.jsx' has a burger menu for easy navigation. 'SearchBar.jsx' handles user messages. 'SubmitButton.jsx' handles login submission. 'RailTicket.jsx' handles outputting the ticket booking results in the form of a UK train ticket.
- Features - 'staffSlice.js' contains the Redux reducer logic for storing the staff login state (Figure 15).

```

MainChatbot.jsx ×
Frontend_scripts > chatbot > src > pages > MainChatbot.jsx > MainChatbot > useEffect() callback
1  import SearchBar from "../components/SearchBar";
2  import Chat from "../components/Chat";
3  import { useState, useId, useEffect, useRef } from "react";
4  import { useSelector } from "react-redux";
5  import axios from 'axios';
6
7  const MainChatbot = () => {
8
9      const [userInput, setUserInput] = useState("");
10     const [userID, setUserID] = useState("")
11     const staff = useSelector((store) => store.staff);
12     const [chats, setChats] = useState([
13         {
14             text : "Hi! I'm Cheep. How can I help you?",
15             sender : "ai"
16         }
17     ]);
18     const websocket = useRef(null);
19     const id = useId()
20     const endRef = useRef(null)
21
22     useEffect(()=>{
23
24         setUserID(id);
25         // open websocket connection
26         websocket.current = new WebSocket(`ws://127.0.0.1:8000/ws/${id}`);
27         websocket.current.onmessage = (event) => {
28
29             // receive chatbot staged response
30             const chatbotResponse = JSON.parse(event.data);
31
32             if (typeof chatbotResponse === "string"){
33                 const nextChatBot = {
34                     text : chatbotResponse,
35                     sender : "ai"
36                 };
37                 setChats((chats) => [...chats, nextChatBot]);
38             } else if (typeof chatbotResponse === "object") {
39                 const nextChatBot = {
40                     text : chatbotResponse,
41                     sender : "ai"
42                 };
43                 setChats((chats) => [...chats, nextChatBot]);
44             }
45         };
46     });
47 }

```

Figure 14: Code example showing some of the code in 'MainChatbot.jsx'. This depicts the states used (lines 9-18), establishing the WebSockets link (lines 24-25) and using spread operation to update the webpage with the AI chat message (line 42). Note that this is performed within a useEffect() hook.

There is a theme control option which is handled through cookies and provided through the DaisyUI plugin for TailwindCSS. This allows the user to select light and dark themed pages. TailwindCSS and DaisyUI were also used for the main web app styling. Moreover, when a staff member logs in, their username is stored as a cookie and displayed on the navigation bar.

```

JS staffSlice.js M x
Frontend_scripts > chatbot > src > features > JS staffSlice.js > [?] default
1  import { createSlice } from "@reduxjs/toolkit";
2  import { toast } from "react-toastify";
3
4  const initialState = {
5    staff : undefined,
6    user : undefined
7  };
8
9  // reducers to deal with admin login action by storing database bearer token
10 // as "staff" in state store.
11 const staffSlice = createSlice({
12   name: "staff",
13   initialState,
14   reducers: {
15     loginStaff: (state, action) => {
16       const accepted = action.payload[0];
17       const username = action.payload[1];
18       if(accepted === true){
19         state.staff = accepted;
20         state.user = username;
21       } else {
22         state.staff = undefined;
23         state.user = undefined;
24       }
25     },
26     logoutStaff: (state) => {
27       state.staff = undefined;
28       state.user = undefined;
29       toast.success("You've logged out!");
30     },
31   },
32 });
33
34 export const { loginStaff, logoutStaff } = staffSlice.actions;
35 export default staffSlice.reducer;

```

Figure 15: Code example from ‘staffSlice.js’ showing the reducer logic for storing the operational staff login token (in this instance a boolean value), that is received from the PostgreSQL database. This allows a staff user to ask queries related to operational events.

The ‘MainChatbot.jsx’ page contains a speech-to-text function which uses the speech recognition webkit through the Web Speech API. When a user input is entered via speech-to-text, this triggers the response to be read aloud using the speech synthesis function of the Web Speech API. This provides an additional layer of accessibility for users.

The application frontend is served using Vite, which is a fast and efficient frontend build tool. In order to maintain a WebSockets connection to the back-end served by FastAPI, a WebSockets connection is established in ‘MainChatbot.jsx’ and requests are posted using Axios for simplicity. This is an asymmetric WebSockets / FastAPI POST method, in that the WebSockets connection is opened in order to return multiple messages from the backend, while the user requests are still made using a POST method to the ‘chat/’ FastAPI endpoint.

5.2 NLP

The NLP component in ‘NLPE.py’ is handled through Wit.ai with a model that was trained on sentences, phrases and words specific to the task, in order that certain intents and entities could be recognised and returned by the model. These intents and entities relate to the core functions of the chatbot: ticket booking and subsequent details, delay prediction, operational events, greetings, farewells and thanks. In this way, the conversation flow can begin, end and relate the goals of the chatbot but nothing else. The chatbot does not handle open-ended conversation or ‘chit-chat’. Figure 16 provides an example of the kinds of phrases used to train the model and how intent and entities were described and extracted.

wit.ai

DocsHelpsamgarland

←cheepttrain

+

Composer

Understanding

Management

Insights

Improve your app

Review text utterances in small batches. Validate predictions to improve the language understanding models.

Understanding review

Add a new utterance

Add a sample utterance and specify an intent. You can also highlight words or phrases in the utterance to annotate.

Utterance ⓘ

“ I need a single ticket from London liverpool street to leeds on the 5th June around 5pm ”

193

Intent ⓘ

booking

Out of Scope

Entity	Role	Resolved value	Confidence	
single_ticket	single_ti...	single	100%	×
ticket	ticket	ticket	100%	×
from	from	from	100%	×
wit/location	location	London liverpool stre	100%	×
to	to	to	100%	×
wit/location	location	Leeds	100%	×
wit/datetime	datetime	2025-06-05T17:00	100%	×

Train and Validate

Figure 16: Screenshot of model training on the Wit.ai dashboard. Here we can see a complex sentence being used to train the model, with an intent (in this case, booking) and several bespoke entities being extracted (single_ticket, ticket, from, location, to, location and datetime). These entities are a mix of Wit.ai entities and bespoke entities that were created for the task.

The Wit.ai model is accessed through an API and the JSON response is stored in a variable ‘user_msg’ which can be unpacked as needed within the software system. Each time a new user message is passed through the WebSockets/FastAPI layer to the backend, a new API call is made to the Wit.ai model to return a structured response.

5.3 Reasoning Engine

The reasoning engine in ‘REngine.py’ is built using Python’s Experta expert system library and consists of rules to handle different kinds of user intent, including ‘greeting’, ‘thanks’, ‘booking’, ‘train_delay’, ‘operational_event’ and ‘goodbye’, which can be seen in Figure 17.

```

REngine.py
Backend_scripts > chatbot_scripts > REngine.py > ChatBot > booking
1  '''
2  This file is the chatbot reasoning engine.
3  '''
4  from experta import *
5  from datetime import datetime
6  from ..chatbot_scripts.booking import BookingQuery
7  from ..chatbot_scripts.train_delay import DelayQuery
8  from ..chatbot_scripts.operational_events import ContingencyEvent, OperationalEvent
9
10 class Intention(Fact):
11     "Thanks, greeting, booking, train_delay, operational_event and goodbye"
12
13 class ChatBot(KnowledgeEngine):
14
15     def __init__(self, user_msg, admin, session_state, confirmation):
16         super().__init__()
17         self.user_msg = user_msg
18         self.admin = admin
19         self.response = {}
20         self.session_state = session_state
21         self.confirmation = confirmation
22         self.subengine_fired_rules = set()
23         self.subengine_final_response = set()
24
25     @Rule(Intention(action="greeting"))
26     def greeting(self):
27         self.response = "Hey! How's it going? Want to ask me something?"
28
29     @Rule(Intention(action="thanks"))
30     def thanks(self):
31         self.response = "My pleasure!"
32
33     @Rule(Intention(action="booking"))
34     def booking(self):

```

Figure 17: Code example from 'REngine.py' showing the reasoning engine code structure. This shows the engine classes, class objects and several rules.

A user session state (a dictionary), which is defined in 'main.py' allows the conversation context to be tracked and dropped as appropriate. Through storing a session state linked to a unique user ID, which is passed from the frontend, 'main.py' stores information about the user intent and the information they have already supplied. This session state is passed between 'main.py', 'REngine.py' and other relevant files to ensure that no user query information is lost and that a consistent conversation is tracked. This can be seen in Figure 18.

```

main.py M x
Backend_scripts > chatbot_scripts > main.py > ...
29     allow_headers=["*"],
30 )
31
32 # typing for the POST body
33 class UserInput(BaseModel):
34     message: str
35     user_id: str
36     admin: Optional[bool] = None
37
38 # dict to track session and websocket connection
39 session_tracker = {}
40 connections = {}
41
42 # establish websocket connection
43 @app.websocket("/ws/{client_id}")
44 async def websocket_connection(websocket: WebSocket, client_id: str):
45     await websocket.accept()
46     connections[client_id] = websocket
47     try:
48         while True:
49             await websocket.receive_text()
50     except WebSocketDisconnect:
51         del connections[client_id]
52
53 # API to handle chatbot functions
54 @app.post("/chat")
55 async def chatbot(user_input: UserInput):
56
57     # get uid
58     uid = user_input.user_id
59     # get admin privileges
60     user_admin = user_input.admin
61     # extract entities etc from user_msg using WIT
62     user_msg = client.get_message(user_input.message)
63
64     if uid not in connections:
65         return JSONResponse(content={"error": "WebSocket not connected"}, status_code=400)
66
67     # connect to websocket
68     websocket = connections[uid]
69     act = ""
70     def create_session_state():
71         session_tracker[uid] = {
72             "action" : None,
73             "info" : {}
74         }
75     return session_tracker[uid]
--

```

Figure 18: Code example from ‘main.py’ (chatbot_scripts) showing the code for the WebSockets connection through a FastAPI endpoint. It also depicts the POST body typing (lines 33-36), as well as how the user message is processed and session state is established (line 62 and lines 70-75, respectively). NB- Line 62 calls the ‘get_message()’ function in ‘NLPE.py’, which makes an API call to Wit.ai to extract intent and entities from the user message.

The combination of a session state and WebSockets allows for multi-turn conversations between the user and the chatbot system, in which the system can track the missing data it needs to fulfil a query and ask the user a series of questions to obtain this data in turn, without losing any data or the context of the conversation. This allows for a more seamless and human-like conversation. The aim of this is to gather the requisite data necessary to trigger the action given in the user intent, such as booking a train ticket or predicting train delay. As a conversation progresses, engine responses are captured and passed back-up to ‘main.py’ as a queue, where the first member of the queue is the chatbot response. Using queue logic allows for extra questions to be inserted at the front of the queue where necessary, and for conditional logic that handles when the queue is empty, so that further actions, such as searching for train tickets or predicting train delay, are triggered. An example of this can be seen in Figure 19.

```

144
145     # if a list is returned from the engine take the first item in the list to add to chat
146     if isinstance(engine.response, list):
147         await websocket.send_json(engine.response[0])
148
149     # if the engine response is final element of booking query list then search for tickets.
150     if engine.response[0] == "Let me look for those tickets for you. This might take a few moments.":
151         searching = "...searching for ticket now..."
152         await websocket.send_json(searching)
153         try:
154             # find the station codes and add to query as origin_code and destination_code
155             query = return_station_code(session_tracker[uid])
156
157             # search for tickets with National Rail datafeed
158             journey_info = extract_journey(query)
159             ticket = journey_info
160             operator_link = find_operator_site(journey_info['outward']['operator'])
161
162             await websocket.send_json(ticket)
163             await websocket.send_json("Link to operating company website: ")
164             if isinstance(operator_link, str):
165                 await websocket.send_json({"LINK": operator_link})
166             elif isinstance(operator_link, list):
167                 for link in operator_link:
168                     await websocket.send_json({"LINK": link})
169             await websocket.send_json("Is there anything else I can help you with?")
170             # resets states
171             engine.session_state = {}
172             session_tracker[uid] = {}
173         except:
174             await websocket.send_json("Oops, something went wrong. Please try the query again.")
175             engine.session_state = {}
176             session_tracker[uid] = {}
177

```

Figure 19: Code example showing the code for processing a user booking request in ‘main.py’. If the response from the engine takes a certain string form, it implies that the booking query is complete and this triggers the system to (1) find and add the station codes to the query using ‘load_station_code.py’ (line 154), (2) access the data feed through ‘feed_access.py’ and return the cheapest ticket (line 158), which is sent back to the chat along with, (4) a link to the ticket booking page on the operating company website using ‘ticket_website.py’ (line 160-168).

5.4 Ticket Booking

The ‘booking.py’ file handles building the booking query through the BookingQuery class and class functions using a question and response pattern built into the class logic. The session state is set to a ‘booking’ action so that the engine knows to return to the booking query after each user response to a question. The session state dictionary also holds ‘info’, which contains keys, initially set to ‘None’, which are redefined based on the user inputs, which can be seen in Figure 20. For instance, if the initial input contains information about the outgoing date of the journey, the class functions will extract these entities from the initial user input and will not ask questions relating to the outgoing date. However, if the user does not specify the outgoing date in the initial input, they will be prompted to input this. This prevents frustrating the user by asking for information they have already provided, but also ensure that all the necessary information to find ticketing information is provided (see: Figures 20 and 21).

```

33     @Rule(Intention(action="booking"))
34     def booking(self):
35
36         if self.session_state["action"] == None:
37             self.session_state["action"] = "booking"
38
39         if len(self.session_state["info"]) == 0:
40             self.session_state["info"]["month_out"] = None
41             self.session_state["info"]["day_out"] = None
42             self.session_state["info"]["time_out"] = None
43             self.session_state["info"]["month_ret"] = None
44             self.session_state["info"]["day_ret"] = None
45             self.session_state["info"]["time_ret"] = None
46             self.session_state["info"]["origin"] = None
47             self.session_state["info"]["destination"] = None
48             self.session_state["info"]["ticket"] = None
49             self.session_state["info"]["adults"] = None
50             self.session_state["info"]["children"] = None
51             self.session_state["info"]["railcard"] = None
52             self.session_state["info"]["card_code"] = None
53
54         missing_list, dict_obj = BookingQuery(self.session_state["action"],
55                                             self.session_state["info"]).extract_query(self.user_msg,
56                                             self.confirmation,
57                                             missing_data=[])
58         self.confirmation = None
59         self.session_state = dict_obj
60         self.response = missing_list

```

Figure 20: Code example showing how a session state is initiated in ‘REngine.py’ (lines 36-52). This also shows how the BookingQuery class is called in ‘booking.py’ (lines 54-57) and how the responses are dealt with (lines 58-60).

```

39     # get outgoing train time
40     def train_datetime_out(user_msg):
41
42         try:
43             if 'datetime' in user_msg['outcomes'][0]['entities']:
44                 if "value" in user_msg['outcomes'][0]['entities']['datetime'][0]:
45                     dateTime_out = user_msg['outcomes'][0]['entities']['datetime'][0]['value']
46                     set_vals_dt(self.info["month_out"], self.info["day_out"], self.info["time_out"], missing_data, dateTime_out, "out")
47                 elif "from" in user_msg['outcomes'][0]['entities']['datetime'][0]:
48                     dateTime_out = user_msg['outcomes'][0]['entities']['datetime'][0]['from']['value']
49                     set_vals_dt(self.info["month_out"], self.info["day_out"], self.info["time_out"], missing_data, dateTime_out, "out")
50             else:
51                 missing_data.append("Could you please enter the train departure day, month and time?")
52         except:
53             missing_data.append("Could you please enter the train departure day, month and time?")
54

```

Figure 21: Code example showing the ‘train_datetime_out()’ function in ‘booking.py’. This demonstrates the mechanism by which the class functions can use extracted entities from the initial user input as well as prompt the user to enter an input, thus reducing the need for the user to re-enter information.

```

189         # calling functions depending on session state
190         if self.info["day_out"] == None:
191             train_datetime_out(user_msg)
192         if self.info["day_ret"] == None:
193             train_datetime_return(user_msg)
194         if self.info["origin"] == None:
195             train_origin(user_msg)
196         if self.info["destination"] == None:
197             train_destination(user_msg)
198         if self.info["ticket"] == None:
199             ticket_type(user_msg)
200         if self.info["ticket"] == "return" and self.info["day_ret"] == None:
201             add_return(user_msg)
202         if self.info["adults"] == None:
203             passengers(user_msg)
204         if self.info["railcard"] == None:
205             railcard_type(user_msg, confirmation)
206         missing_data.append("Let me look for those tickets for you. This might take a few moments.")
207         return missing_data, self.__dict__

```

Figure 22: Code example showing the query builder in ‘booking.py’. This looks for ‘None’ values in the session state dictionary and calls the relevant class functions as required.

Prompts for missing information are appended to an array which acts as a queue, the first element of which is passed back through the WebSockets connection to the chat. When the last element of the array becomes the first element, this prompts ‘main.py’ to trigger searching for and returning the ticket information. Hence, when the user has answered all the relevant questions from the BookingQuery class functions, the final element is passed back through WebSockets to the chat, and further functions are triggered to retrieve ticket information and the link to the ticketing website, as described above.

5.5 Delay Prediction

If the user intent is ‘train_delay’, ‘REngine.py’ creates a session state and calls the ‘train_delay.py’ DelayQuery class in much the same way outlined in Section 5.4 regarding ticket booking. Similarly, this prompts the user to enter further information, if necessary, to build a query. Upon completion of the user query, the ‘predict_journey()’ function in ‘journey_predict.py’ is triggered, which makes an API call to the ‘train-prediction-api’ package, which serves as the delay prediction and contingency plan module.

```

84 # Prediction endpoint
85 @app.post("/journey-predict", response_model=JourneyDelayResponse)
86 def predict_journey_delay(request: DelayRequest):
87     try:
88         if not request.current_station_code or not request.target_station_code:
89             raise HTTPException(status_code=400, detail="Both current_station_code and target_station_code are required")
90
91         try:
92             start_index = station_sequence.index(request.current_station_code)
93             end_index = station_sequence.index(request.target_station_code)
94         except ValueError:
95             raise HTTPException(status_code=400, detail="Invalid station codes in route")
96
97         if start_index >= end_index:
98             raise HTTPException(status_code=400, detail="Target station must come after current station in route")
99
100         path = station_sequence[start_index:end_index + 1]
101
102         delays = {}
103         current_delay = request.previous_arrival_delay
104
105         for i in range(len(path) - 1):
106             curr_station = path[i]
107             next_station = path[i + 1]
108             try:
109                 station_index = float(station_index_map[curr_station])
110                 station_day_interaction = station_index * request.day_of_week
111             except KeyError:
112                 raise HTTPException(status_code=400, detail=f"Station code {curr_station} not found in station_index_map.")
113
114             input_data = pd.DataFrame([[
115                 'day_of_week': request.day_of_week,
116                 'is_weekend': request.is_weekend,
117                 'actual_travel_time_to_next': request.actual_travel_time_to_next,
118                 'hour_of_day': request.hour_of_day,
119                 'is_peak_hour': request.is_peak_hour,
120                 'is_holiday': request.is_holiday,
121                 'station_index': station_index,
122                 'is_first_station': 1 if i == 0 else 0,
123                 'is_last_station': 1 if i == len(path) - 2 else 0,
124                 'previous_arrival_delay': current_delay,
125                 'cumulative_delay': current_delay,
126                 'prev_delay_1': current_delay,
127                 'prev_delay_2': 0.0,
128                 'station_day_interaction': station_day_interaction
129             ]])
130
131             predicted_delay = cat_model.predict(input_data)[0]

```

Figure 23: Code example from ‘train-prediction-api’ package ‘main.py’, showing the FastAPI ‘/journey-predict’ endpoint. This combines efficient error handling with the train delay prediction model.

The delay prediction model is written in Python and served from a FastAPI back-end through the ‘train-prediction-api’ package’s ‘main.py’ (Figure 23). The FastAPI back-end provides a lightweight, scalable and accessible interface to predict train delays for specific station-to-station segments. It leverages a segment-wise CatBoost regression model to make real-time predictions based on user-provided journey details. It accepts structured inputs via the FastAPI ‘/journey-predict’ endpoint, processes them through pre-trained models, and returns predicted delays for specific station segments.

Significant detail is given on data preprocessing, model testing and evaluation in Section 3.1.5.

5.6 Operational Events (Contingency Plan Retrieval)

The ‘operational_events.py’ file contains an engine built as an Experta expert system. This acts as a subengine to the main chatbot reasoning engine in ‘REngine.py’. If a staff user is logged-in and their intent is to retrieve contingency plan information, ‘REngine.py’ creates a session state and calls the ‘operational_events.py’ subengine. Within the operational events subengine, the rules are triggered as outlined in Section 3.1.6. The order of rules being triggered is controlled by rule salience and using ‘engine.declare()’ statements to trigger knock-on or cascading rules. Figure 24 gives an example of an engine rule. Engine halting is used in class functions where the user is prompted to enter further

information to build the operational event query. This is necessary given that the engine response needs to be passed back to the main reasoning engine before being passed to the WebSockets FastAPI endpoint in ‘main.py’ (in: Backend_scripts/chatbot_scripts/).

```

64 # ==== This section handles location and knock-on effects ==== #
65
66 @Rule(Event(location=None), salience=15)
67 def handle_location(self):
68
69     line = []
70     encoded = ""
71
72     if 'location' in self.user_msg['outcomes'][0]['entities']:
73         for i in range(len(self.user_msg['outcomes'][0]['entities']["location"])):
74             if "value" in self.user_msg['outcomes'][0]['entities']["location"][i]:
75                 line.append(self.user_msg['outcomes'][0]['entities']["location"][i]["value"])
76                 code = return_station_code(line[i])
77                 encoded += code
78             encoded = encoded[:3] + "_" + encoded[3:]
79             self.session_state["info"]["location"] = encoded
80             self.session_state["fired_rules"].append("handle_location")
81             self.declare(OperationalEvent(event=self.session_state["info"]["event"]))
82     else:
83         self.missing_data.append("Please can you tell me which stations the event is between?")
84         self.halt()

```

Figure 24: Code example from ‘operational_events.py’ showing an expert system rule decorating a class function. In this instance, the rule handles the case in which the staff user has given no event location. When the event location has been given, it is added to the session state and the engine declares the next rule with the relevant session state value (lines 79-81). Otherwise, the function prompts the user for input and halts the engine (line 84-84).

Where further contingency plan information is required, rules call the ‘get-contingency-plan’ FastAPI endpoint in the ‘train-prediction-api’ package ‘main.py’ file (Figure 25). Through the FastAPI interface, user inputs are transformed into natural-language queries, embedded in real-time (using the same OpenAI model given in Section 3.1.6) and matched against the FAISS indices. Once the top matches are returned, the system performs detailed parsing of the Metadata, pulling out structured fields by splitting text on known delimiters, removing irrelevant prefixes and applying regex patterns to extract critical information such as cancelation lists, special notes and distinctive operational instructions. There are a set of domain-specific business rules through which this parsed data is then passed. These rules segregate train services based on time-of-day such as AM peak, PM peak or off-peak, evaluate delay windows and determine which trains to include or exclude based on the factors like historical disruption frequency or operational impact.

From ‘livst-nrch-list.json’, we further integrate live timetable data in order to provide precise scheduling context, synchronising disruption responses with real train operations. A clean and structured JSON response is the final output that merges static policy data with live operational intelligence. The system connecting to the ‘get-contingency-plan/’ endpoint, is modular, semantically rich and capable of adapting to a wide variety of user queries and rail disruption scenarios such as providing quick, reliable and context aware responses without hardcoded lookup logic. It is also capable of adding more information to enhance the knowledge base which we believe is paramount for an efficient AI based automated operational event system.

```

409 # Stop Orders Handler
410
411 def handle_stop_orders(request):
412     if len(request.location) != 2:
413         raise HTTPException(status_code=400, detail="Location must include two stations")
414
415     query = f"{request.type} disruption near {request.location[0]} or {request.location[1]} GE Main Line"
416     results = search_faiss_d2(query)
417
418     if not results:
419         return {"status": "not_found", "message": "No relevant stop orders found."}
420
421     return {
422         "status": "success",
423         "query": query,
424         "top_matches": results
425     }
426
427 # ----- Main Route -----
428 @app.post("/get-contingency-plan")
429 def get_contingency_plan(request: ContingencyRequest):
430     dispatch_table = {
431         "running_as_booked": handle_running_as_booked,
432         "stop_orders": handle_stop_orders
433     }
434
435     service_text = request.service_type.lower()
436
437     # Use regex-based keyword inference
438     if "running_as_booked" in service_text or re.search(r"\brunning\b|\bbooked\b", service_text):
439         handler_key = "running_as_booked"
440     elif "stop_orders" in service_text or re.search(r"\bstop\b|\bstoppers\b|\bstop orders\b", service_text):
441         handler_key = "stop_orders"
442     else:
443         raise HTTPException(status_code=400, detail=f"Unsupported service_type: {request.service_type}")
444
445     return dispatch_table[handler_key](request)

```

Figure 25: Code example showing the ‘/get-contingency-plan’ endpoint (lines 428-429) and the function to handle retrieving special stop orders (lines 411-425) from ‘train-prediction-api/app/main.py’

As the engine cascades through the rules in ‘operational_events.py’ a final response of contingency plan information is build in the ‘final_response’ array. Once the sub-engine has completed, the final response is returned to the chat (Figure 26). This contains all the appropriate information relating to the staff user query.

```

348
349 @Rule(FinalResponse(action="trigger"), salience=0)
350 def push_response(self):
351     final_response = "\n".join(self.session_state["final_response"])
352     final_response += "\n\nIs there anything else I can help you with today?"
353     self.missing_data.append(final_response)
354     self.session_state = {}
355     self.halt()

```

Figure 26: Code example from ‘operational_events.py’ showing the rule and class function to trigger sending the aggregated final response from the sub-engine back to the chat.

5.7 Knowledge Base

The knowledge base consists of expert system rules, vector files and stored JSON data (also including vector embeddings). The expert system rules have been discussed in detail in Section 3.1.6. The following section details other elements of the knowledge base.

5.7.1 JSON files

Station names for every UK rail station (stored in 'codes.json') and railcard types (stored in 'cards.json'), were transformed into vector embeddings using the BERT 'all-MiniLM-L6-v2' transformer model through the Python 'sentence_transformers' library, an example of which can be seen in Figure 27. The resultant vector embeddings were stored in 'station_embeddings.json' and 'card_embeddings.json', respectively. JSON storage was used given the relatively small number of station names and very small number of railcard types. 'load_station_codes.py' and 'find_railcard.py' are used to extract the relevant information from the vector embedding files using cosine similarity scores, as shown in Figure 28.

```
6 import json
7 from sentence_transformers import SentenceTransformer
8
9 with open("Backend_scripts\\chatbot_scripts\\Assets\\codes.json", "r") as f:
10     data = json.load(f)
11
12 station_list = list(data.keys())
13 model = SentenceTransformer('all-MiniLM-L6-v2')
14 vectors = model.encode(station_list)
15 station_embeddings = {station: vector.tolist() for station, vector in zip(station_list, vectors)}
16
17 with open("Backend_scripts\\chatbot_scripts\\Assets\\station_embeddings.json", "w") as g:
18     json.dump(station_embeddings, g)
19
```

Figure 27: Code example from 'embed_station_codes.py' showing the BERT 'all-MiniLM-L6-v2' transformer model from the Python 'sentence_transformers' library being used to transform UK station names into vector embeddings.

```

12 def return_station_code(query):
13
14     model = SentenceTransformer('all-MiniLM-L6-v2')
15
16     codes_path = "Backend_scripts\\chatbot_scripts\\Assets\\codes.json"
17     with open(codes_path) as f:
18         codes = json.load(f)
19
20     embeddings_path = "Backend_scripts\\chatbot_scripts\\Assets\\station_embeddings.json"
21     with open(embeddings_path) as g:
22         embeddings = json.load(g)
23
24     def find_station_code(input, model, codes, embeddings):
25
26         station = input.lower()
27         station = station.title()
28
29         station_list = list(codes.keys())
30         vectors = list(embeddings.values())
31
32         embed_query = model.encode(input)
33         scores = cos_sim(embed_query, vectors)[0]
34
35         best_match = scores.argmax()
36         best_score = scores[best_match].item()
37         closest = station_list[best_match]
38
39         if best_score >= 0.60:
40             code = codes[closest]
41             return code
42         else:
43             return input
44
45     if isinstance(query, str):
46         query = find_station_code(query, model, codes, embeddings)
47     if isinstance(query, dict):
48         if "origin" in query["info"].keys():
49             query["info"]["origin_code"] = find_station_code(query["info"]["origin"], model, codes, embeddings)
50         if "destination" in query["info"].keys():
51             query["info"]["destination_code"] = find_station_code(query["info"]["destination"], model, codes, embeddings)
52         if "current_station" in query["info"].keys():
53             query["info"]["current_station_code"] = find_station_code(query["info"]["current_station"], model, codes, embeddings)
54
55     return query

```

Figure 28: Code example from ‘load_station_code.py’ showing cosine similarity search being used (lines 33-37), to find the best station match between the user input and the embedded vectors.

The ‘operators.json’ file contains all UK rail operators and ticketing websites. The ‘ticket_websites.py’ file uses this to return the operating company ticket website.

In ‘train-prediction-api/app’, the ‘json’ and ‘output’ folders contain JSON files used to store extracted operational event and contingency plan information used to give the textual response from the ‘get-contingency-plan/’ endpoint.

5.7.2 Numpy Vector Files

As mentioned in Section 3.1.6, OpenAI was used for text-extraction and vectorisation of the relevant contingency plan information held in the historic pdfs (D1-D3). The ‘.npy’ vector files are stored in ‘train-prediction-api/app/output’, as they are accessed from the package ‘main.py’ file when the ‘get-contingency-plan/’ endpoint is called.

6 Testing

6.1 Unit Testing

Unit testing was carried-out using Python’s Unittest library on the chatbot backed and can be found in the ‘Backend_scripts/tests’ folder. Unit tests cover both util functions and engine class functions, examples of which can be seen in Figures 29 and 30, respectively. The completed test runner console output can be seen in Figure 31, demonstrating the tests all passed successfully. Unittest was used because it is a well-supported and built-in Python library that is simple to use.

```

23     # testing load_station_code.py function
24     def test_return_station_code(self):
25
26         test_1 = {
27             "info" : {
28                 "origin" : "leeds",
29                 "destination" : "sheffield"}}
30
31         expected_output_1 = {
32             "info" : {
33                 "origin" : "leeds",
34                 "destination" : "sheffield",
35                 "origin_code" : "LDS",
36                 "destination_code" : "SHF"}}
37
38         test_2 = {
39             "info" : {
40                 "origin" : "norwich",
41                 "destination" : "london liverpool street"}}
42
43         expected_output_2 = {
44             "info" : {
45                 "origin" : "norwich",
46                 "destination" : "london liverpool street",
47                 "origin_code" : "NRW",
48                 "destination_code" : "LST"}}
49
50         self.assertEqual(return_station_code(test_1), expected_output_1)
51         self.assertEqual(return_station_code(test_2), expected_output_2)
52

```

Figure 29: Unit test for 'return_station_code()' function in 'load_station_code.py', using an assertEquals test form.

```

157     # testing REngine.py 'train_delay' intention with semi_formed
158     # session state
159     def test_semiformed_train_delay(self):
160         engine = ChatBot({"outcomes" : [{
161             "entities" : ""}],
162             False,
163             {"action" : "train_delay",
164              "info" : {
165                  "month_out" : "June",
166                  "day_out" : 12,
167                  "time_out" : "18:00:00",
168                  "origin" : "Leeds",
169                  "destination" : "York",
170                  "current_station" : "Huddersfield",
171                  "current_delay" : None
172              }},
173             None)
174         engine.reset()
175         engine.declare(Intention(action="train_delay"))
176         engine.run()
177         resp = "If available, please enter your current delay in minutes:"
178         not_resp = "Where are you travelling to?"
179         assert resp in engine.response[0]
180         assert not_resp not in engine.response[0]
181

```

Figure 30: Unit test for a semi-formed train delay query entering ‘REngine.py’ using the assert in and assert not in test forms.

```

PS C:\Users\samue\OneDrive\Desktop\AI module\Ch2\ChatBot> & C:/Users/samue/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/samue/OneDrive/De
test_find_operator_site (__main__.TestCases.test_find_operator_site) ... ok
test_intention_booking (__main__.TestCases.test_intention_booking) ... ok
test_intention_operational_event (__main__.TestCases.test_intention_operational_event) ... WARNING:experta.watchers.ENGINE:Declaring fact before reset()
ok
test_intention_train_delay (__main__.TestCases.test_intention_train_delay) ... ok
test_return_railcard_code (__main__.TestCases.test_return_railcard_code) ... ok
test_return_station_code (__main__.TestCases.test_return_station_code) ... ok
test_semiformed_train_delay (__main__.TestCases.test_semiformed_train_delay) ... ok

-----
Ran 7 tests in 9.206s

OK
PS C:\Users\samue\OneDrive\Desktop\AI module\Ch2\ChatBot>

```

Figure 31: Console output from running the unit tests in ‘test_core.py’ showing all tests successfully passed.

6.2 Integration Testing

Integration testing was carried-out on the ‘train-prediction-api’ package endpoints. Postman was used to send real API calls in order to test whether components from the backend and different packages interact properly together when combined. Screenshots of the Postman testing tool, used to test the staff user login API, the train delay prediction API, and the contingency plan API endpoints can be seen in Figures 32, 33 and 34, respectively. Postman was used as it offers a simple environment to send real rather than mock API calls. This is beneficial, as it allows developers to inspect the genuine API response to ensure they are achieving the desired result.

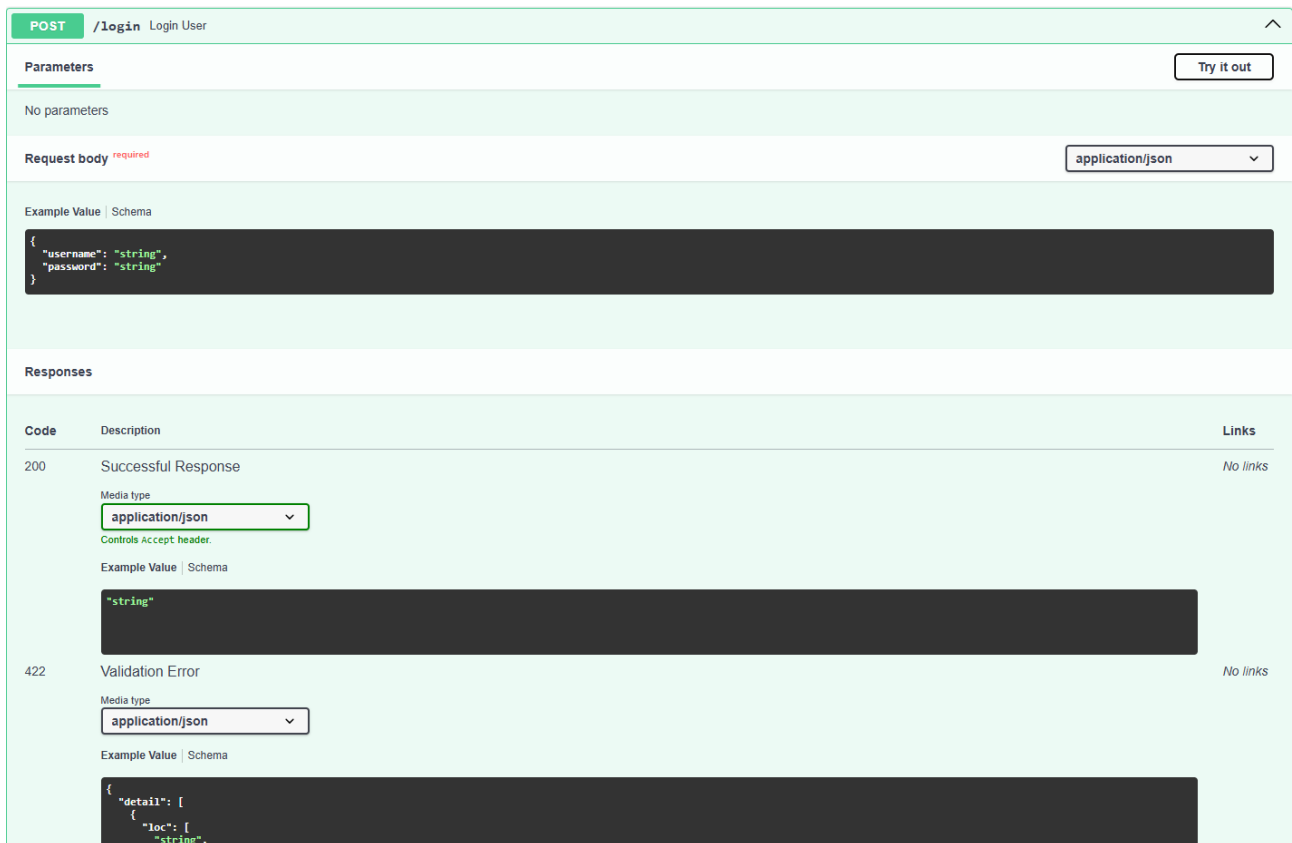


Figure 32: Screenshot of the Postman API testing tool, showing the staff login API endpoint.

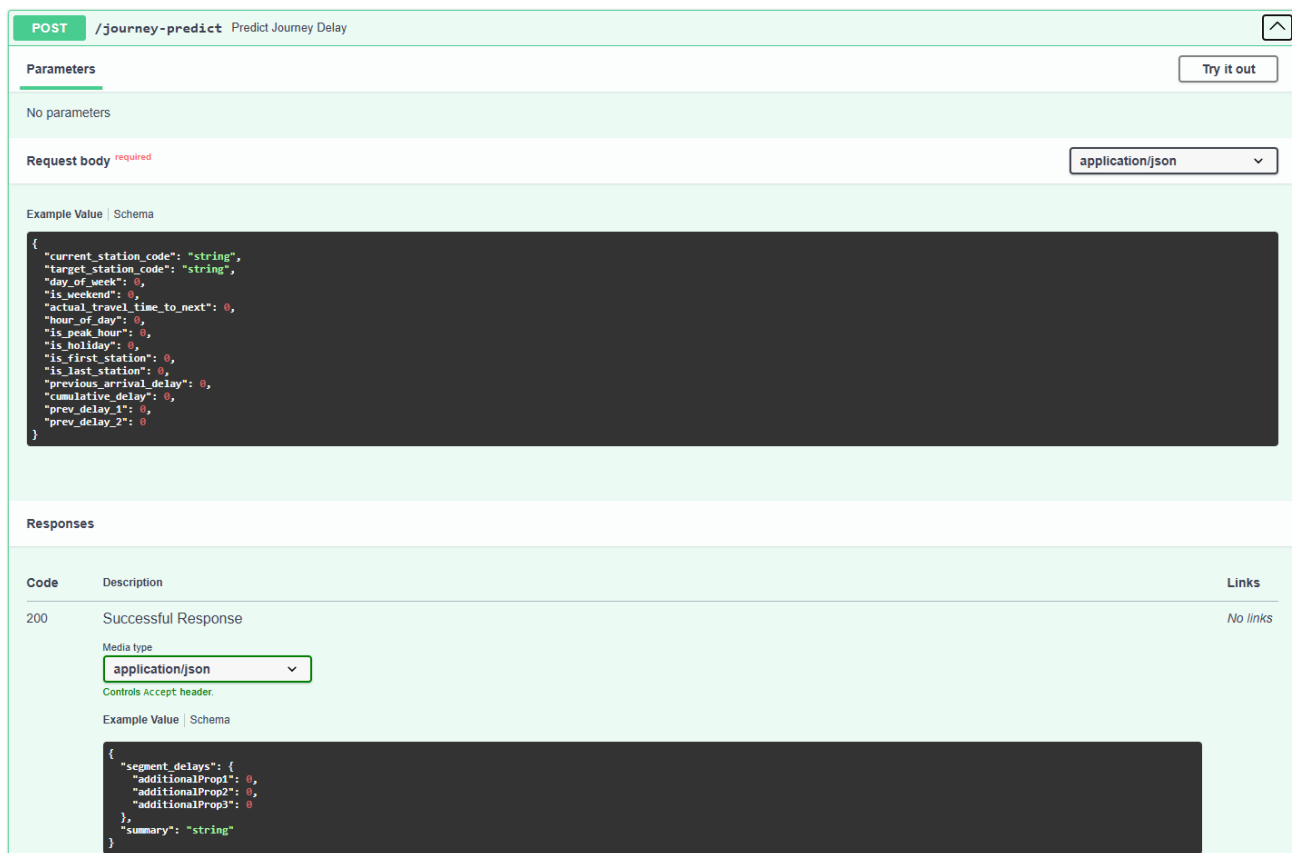


Figure 33: Screenshot of the Postman API testing tool, showing the train delay prediction API endpoint.



Figure 34: Screenshot of the Postman API testing tool, showing the response body and 200 code after testing the contingency plan API endpoint.

6.3 System and Usability Testing

Full system and user testing were carried-out throughout the development process to ensure correct flow of processes, user goals were fulfilled and the software packages were integrated effectively. We also used systematic usability testing to test whole system processes and edge cases. To ensure a reasonable degree of fairness in the testing, the system usability tests were written and input by student 10049374, who was not involved with the development of task 1. This allowed for user cases to not be biased by the specific language use of the developer. Table 5 shows the results of the first round of usability testing in order to give the reader a sense of how testing was carried-out and how unsuccessful test-cases were recorded and rectified.

ID	Scenario Description	Test Inputs	Expected Results	Actual Results	Status	Comment
1.	One-way trip: Norwich to London, 15 July	From: Norwich / To: London Date: 15 August	Display cheapest ticket with booking link	Sorry, you can't book a train this far in the UK. Please enter a date and time within the next two months	PASSED	done
2.	Return trip: Norwich → London → Norwich (2-day gap)	I need a train ticket from Norwich to London on the 16th of June returning on the 20th of June at 5pm	Show cheapest return ticket and booking link	Do you want a single or return ticket?	PASSED	It understands the return, but asking for a confirmation

ID	Scenario Description	Test Inputs	Expected Results	Actual Results	Status	Comment
3.	Complex return: Norwich to Oxford, time preferences	I need a ticket on 20 June before 11am returning on 30 June after 1pm	Filter based on time, return cheapest options	Return journey must be after the outgoing journey	FAILED	This is an issue if the NLPU – see wit.ai
4.	Off-peak ticket search	I want a ticket from Cambridge to Manchester on 18th May at Off-peak time	Show cheapest off-peak option	We continued through and got the cheapest ticket at 1pm	PASSED	none
5.	Student railcard discount	I need a ticket from Leeds to Birmingham with a railcard	Ticket options with railcard discount	The system crashed	FAILED	Can't find the type railcard in the find_railcard file
6.	Group booking	I want a ticket From Norwich to York for 3 adults on tomorrow at 5pm	Show cheapest group options	It worked fine. Showed the cheapest tickets for 3 adults	PASSED	none
7.	Open return ticket	I want an open return ticket From London to: Liverpool	Display open return pricing	Showed advanced standard class ticket	PASSED	none
8.	Same-day round trip	I want a same day round-trip ticket from Norwich to: Peterborough	Cheapest same day return options	Provided the tickets	PASSED	We still had to enter the departure and arrival times. It didn't understand the "same-day round trip". We still need to complete the query
9.	Train delayed at Diss, want updated arrival at London Liverpool Street	My train is delayed, I want to know the arrival time at London Liverpool Street	Go through to the delay module	Unsure of user query	FAILED	Needs to understand that this is about train delay

ID	Scenario Description	Test Inputs	Expected Results	Actual Results	Status	Comment
10.	Onboard train with severe delays, wants impact info	The train from Norwich to London Liverpool Street is at Diss now. It is delayed by 30 minutes. When will it arrive?	Go through to the delay module	Prediction given as 24.09 minutes	PASSED	none
13.	Train has departed late, want arrival forecast	Departure of the train from London to Norwich is delayed for 15 mins. When will it arrive at London?	Go through to the delay module	Could you please enter the planned train departure time?	PASSED	none
14.	Blockage: Colchester-Manningtree	The track from Colchester to Manningtree is blocked.	Go through to ask about event type in operational event subsystem	Prompt to login. When logged-in: Is the line blockage partial or full?	PASSED	none
15.	Following 14. - Line fully blocked	The line is fully blocked	Go through to ask about current delay	What is the current line delay?	PASSED	none
16.	Following 15. – Add current delay	20 mins	Receive contingency plan info	The contingency plan info was given but it was the wrong info - for trains from 06.40am-07.14am	FAILED	The operational events side works, but it retrieves the wrong info through the API.

Table 5: Table showing usability test script with results from the first round of usability testing. This shows both tests cases that passed and failed, with the reason for failure and recommended adjustments given in the comment column.

7 Evaluation

The following section details our AI Chatbot software system evaluation. For details on our train journey delay prediction model evaluation, see **Results and discussion** in Section 3.1.5.

7.1 Chatbot Performance

Application performance was tested through the main chatbot interface WebSockets connection using the ‘/chat’ FastAPI endpoint. The results of this evaluation can be seen in Table 6 and the Python script can be found in ‘Performance.evaluation/performance.py’. This file connects to the WebSockets

endpoint in ‘Backend_scripts/chatbot_scripts/main.py’ and uses this to post a message to the ‘/chat’ endpoint and receive a WebSockets response (asymmetric WebSockets strategy).

Message ID	Message	Response	Response Time (seconds)
01	‘Hi’	‘Hey! How’s it going? Want to ask me something?’	2.632s
02	‘Can I book a ticket please?’	‘Could you please enter the train departure day, month and time?’	1.516s
03	‘I need train delay info’	‘Could you please enter the planned train departure time?’	1.117s
04	‘I want a contingency plan’	‘Sorry, only operational staff can access contingency plan information. Please login via the dropdown menu to access this function. Otherwise, is there anything else I can help you with?’	0.718s
05	‘thanks’	‘My pleasure!’	0.419s

Table 6: Table showing message response times through the main FastAPI/WebSockets connection in ‘main.py’.

Table 6 demonstrates a reasonable response time given the NLP model access and scale of the application. Long-standing and industry-norm application response time categories set our application between an ‘immediate’ and ‘continuous’ response time (e.g. between 1 and 5 seconds), which is considered to be sufficient for dialogue with a user (Cornejo et al. 2024).

7.2 NLP Accuracy

We used the Wit.ai dashboard to test NLP intent and entity accuracy. Table 7 shows the results of this evaluation, demonstrating high intent and entity F1-scores across most cases. This provides strong evidence for the efficacy of our approach in using Wit.ai over Python libraries for NLP tasks. However, there are clearly some areas where the model could be improved. In particular, the model occasionally confuses location names and passenger numbers, possibly due to conflicting entities. Given more time, we would train the model to recognise all UK rail stations. We would also redefine the preset ‘wit/number’ entity.

ID	Input	Wit.ai Response + Confidence Score	Pass/Fail
1	I need to book a train ticket to Leeds from Norwich on the 4th of June at 4pm.	Intent: booking. Entities: to 100%, ticket 100%, wit/location (Leeds) 94%, from 100%, wit/location (Norwich) 97%, wit/datetime (2025-06-04T16:00) 100%.	PASS
2	I want a ticket from York to Sheffield next Friday around midday for 1 adult and 2 children.	Intent: booking. Entities: single_ticket 100%, from 100%, wit/location (York) 99%, to 100%, wit/location (Sheffield) 93%, wit/datetime (2025-05-30T12:00) 100%, no_adults (1 adult) 100%, no_children (2 children) 100%.	PASS
3	Ticket to London Liverpool St tomorrow at approximately 10 in the morning with a senior rail-card please.	Intent: booking. Entities: ticket 100%, to 100%, wit/location (London) 100%, wit/location (Liverpool St) 54%, wit/datetime (2025-05-21T10:00) 100%, railcard (Senior Railcard) 100%.	FAIL – slightly wrong destination due to bad input.

ID	Input	Wit.ai Response + Confidence Score	Pass/Fail
4	I need a return ticket to Bath from Bristol Temple Meads departing on the 12th June around 9am and returning on the 17th June after 5pm for 1 adult with a 16–25 railcard.	Intent: booking. Entities: return_ticket 100%, to 100%, wit/location (Bath) 100%, from 100%, wit/location (Bristol Temple) 100%, wit/location (Meads) 50%, wit/datetime (2025-06-12T09:00) 100%, returning 100%, wit/datetime (2025-06-17T17:00) 100%, no_adults (1 adult) 100%, railcard (16–25 railcard) 100%.	PASS – cosine similarity may correct slight location error.
5	Cheers, see ya	Intent: goodbye.	PASS
6	stop	Intent: exit_transaction.	PASS
7	My train to Doncaster seems to be delayed	Intent: train_delay. Entities: to 100%, wit/location (Doncaster) 96%.	PASS
8	I want contingency plan info	Intent: operational_event.	PASS
9	I want to find out what to do in an operational event situation.	Intent: operational_event. Entities: to 100%.	PASS
10	No, thanks	Intent: wit/negation. Entities: negation (no) 100%.	PASS

Table 7: Table showing the results of NLP evaluation on our Wit.ai model. The Wit.ai response shows the intents, entities, and values and confidence scores (where given).

7.3 Scalability

The Python Locust framework and dashboard was used to evaluate the scalability of the ‘/journey-predict’ FastAPI endpoint in the ‘train-prediction-api’ package. The Python script can be found in ‘Performance_evaluation/locustfile.py’. We chose this component of the software system specifically for scalability load-testing, given that it requires a prediction model to be run, which can cause a computational bottleneck. We load-tested the API endpoint with as many as 10 concurrent users, with 1 extra user added every second. Figure 35 shows that this test produced no failures after 52 requests with a 10 user max concurrency, which suggests that this component of the software system would scale well.

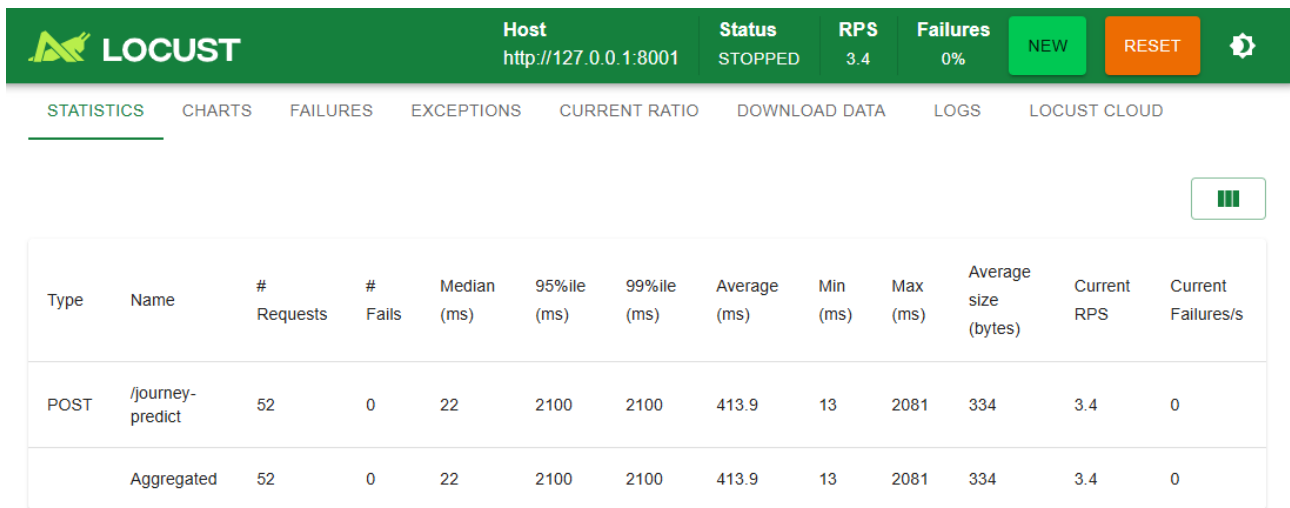


Figure 35: Figure showing the Locust load-testing dashboard while evaluating the ‘/journey-predict’ FastAPI endpoint in the ‘train-prediction-api’ package. This shows no failures after 52 requests, suggesting that the train prediction component of the software system would scale well.

7.4 Testing and Evaluation Summary

The testing and evaluation of our chatbot demonstrates a robust and scalable software system. All unit tests, integration tests and load tests performed were passed without issue. The evaluation of the chatbot performance was also acceptable, with our principal chatbot FastAPI/WebSockets endpoints giving responses in 0-3 seconds. Although, the integration and scalability tests suggest that the software system would scale well, more testing is required in this respect. In particular, passing tests that load all the API endpoints in the system simultaneously would provide strong evidence of a useful system. Usability tests were mostly passed, with some failures recorded. As usability testing was carried-out prior to finishing the project, all the failed cases are rectified in the final software system. However, it is worth noting the importance of effective full system and usability tests. If there were additional time to work on the project, we would expand this method of testing to take into account more edge-cases and potentially unforeseen inputs. In addition, assessment of the Wit.ai model in our NLP accuracy testing led to some failures. As these are issues regarding the model, further work in this regard should focus on broader and more systematic model training, to ensure, for instance, that all possible UK rail station names are recognised as ‘wit/location’ entities.

8 Discussion

The following sections discuss the strengths and limitations of how we developed and implemented our AI chatbot software system, as well as possible future work which could be undertaken to improve the system.

8.1 Strengths

We have demonstrated a robust approach to designing and implementing our AI chatbot software system, which leverages a wide range of different computing techniques, libraries, frameworks, tools and other resources, and has performed well under testing. The system successfully delivers the objectives set-out in the initial coursework brief: (1) The system successfully uses the National Rail Online Journey Planner data-feed to return the cheapest available rail ticket to the user, alongside a link to the ticket purchasing website. The booking component of our system makes complex bookings, allowing the initial user query to include all or none of the booking information required to build a query, so that the user need not repeat themselves. (2) The system also successfully supplies train journey delay prediction through the ‘train-prediction-api’ package, which effectively informs the user of the line delay and expected arrival time of their train between Norwich and London Liverpool

Street. We believe that our train delay prediction component demonstrates a well-reasoned approach, used to solve an interesting and important real-world problem. Through a combination of hand-crafted features, preprocessing techniques and model selection, we were able to select a model using a segment-wise approach, which exhibits strong performance with a RMSE score of 0.50 and an R^2 score of 0.73. (3) Moreover, the system allows operational staff to login and retrieve contingency plan information regarding rail blockage events between Norwich and Colchester. This is successfully served through the ‘train-prediction-api’ package ‘/get-contingency-plan’ endpoint, which leverages FAISS to search vector files and extract relevant contingency information. Finally, the chatbot user interface is simple, accessible and device reactive, allowing use from a range of different kinds of users in diverse situations. It has a staff login function to allow only staff users to access contingency plan information.

8.2 Limitations and Future Work

A key limitation of our software system is that it does not allow the user to purchase a ticket within the system, or begin the booking process for a train ticket on an operator’s website. Future work could include adding a direct link to the train ticket with the option to pay through the user interface. This was not possible to achieve within the project time-frame and with the resources available, as there are dozens of UK ticketing websites, each with different rules around web scraping. Each website would need a different web scraping function and written permission to use headless browsers on the site. This is, therefore, a tricky task both legally and programmatically.

A significant limitation of our train journey delay prediction component is the lack of recent delay data used to train the model, given that the data used is some years out of date. Using more recent data would allow for contemporary line issues, change in line operators, change in weather patterns, amongst others, to be modelled more effectively. It is also possible that the amount of data used to train our models was insufficient for highly accurate delay prediction.

A related area of future work might be to implement advance train delay prediction. This would use a range of available data sources to predict in advance, how likely a train is to be delayed and what alternative forms of transport might be the most appropriate for a user to reach their desired destination within a certain time-frame. A combination of advanced and real-time delay prediction would make an overall more useful system. Furthermore, with regards to the train delay prediction component of our software system, further work could include adding functionality to predict journey delay across the UK. As this is a complex and interconnected problem, AI techniques could be used in the first instance to model UK train delay interconnectivity.

Another limitation of our system is an inability to retain user message information beyond the current context, and a further area for future work might be adding a database for user conversation history. Unsure of the ethical implications of storing customer chat histories, we decided in the design phase not to include this functionality in our chatbot. However, there are cases in which a stored conversation history would be of benefit to the customer, such as after a loss of internet connection, or for a repeat booking made frequently by the user.

Lastly, although we followed the brief for task 3 in the design and implementation of our operational event component, a further limitation of this functionality is that it is limited to line sections between Norwich and Colchester. Further work could enhance the operational events subsystem for extracting contingency plan information by extending this to other rail networks within the UK. This task was beyond the scope of this project given the complexity of contingency plan information across multiple rail lines and networks. However, developing an application to aid operational staff in this regard could save significant resources and have positive implications for customer service.

9 Summary

In summary, this technical report describes in detail the successful design, implementation, delivery and evaluation of our AI chatbot software system. We consider the project to have been successful, and it has left significant room for further future work.

References

- Abedin, A. F., Mamun, A. I. A., Nowrin, R. J., Chakrabarty, A., Mostakim, M. & Naskar, S. K. (2022), ‘A deep learning approach to integrate human-level understanding in a chatbot’, *arXiv preprint arXiv:2201.02735*.
URL: <https://arxiv.org/abs/2201.02735>
- Adamopoulou, E. & Moussiades, L. (2020), An overview of chatbot technology, in I. Maglogiannis, L. Iliadis & E. Pimenidis, eds, ‘Artificial Intelligence Applications and Innovations’, Springer International Publishing, Cham, pp. 373–383.
- Chizhik, A. V. & Zherebtsova, Y. (2020), Challenges of building an intelligent chatbot, in ‘Intelligent Memory Systems’.
URL: <https://api.semanticscholar.org/CorpusID:232116755>
- Cornejo, O., Briola, D., Micucci, D., Ginelli, D., Mariani, L., Santos Parrilla, A. & Juristo, N. (2024), ‘A family of experiments about how developers perceive delayed system response time’, *Software Quality Journal* **32**(2), 567–605.
- IBM (2021), ‘What is a chatbot?’, <https://www.ibm.com/think/topics/chatbots>. Accessed: 2025-04-23.
URL: <https://www.ibm.com/think/topics/chatbots>
- Lin, C.-C., Huang, A. Y. & Yang, S. J. (2023), ‘A review of ai-driven conversational chatbots implementation methodologies and challenges (1999–2022)’, *Sustainability* **15**(5), 4012.
- Luo, B., Lau, R. Y., Li, C. & Si, Y.-W. (2022), ‘A critical review of state-of-the-art chatbot designs and applications’, *Wiley Interdisciplinary Reviews: Data Mining and Knowledge Discovery* **12**(1), e1434.
- Mohammed, M. & Aref, M. M. (2022), ‘Chatbot system architecture’.
URL: <https://arxiv.org/abs/2201.06348>
- Radziwill, N. M. & Benton, M. C. (2017), ‘Evaluating quality of chatbots and intelligent conversational agents’, *arXiv e-prints*.
URL: <https://arxiv.org/abs/1704.04579>
- Stoate, J. (2025), ‘Uk: Lner begins implementation of new delay prediction tool’. Accessed: 2025-04-28.
URL: <https://railway-news.com/uk-lner-begins-implementation-of-new-delay-prediction-tool/>
- Wang, W. (2018), ‘Artificial intelligence modules (cmp6040, 7028) coursework specification’.